Teradata® Vector Store User Guide

Geissendoerfer, Rainer

Rainer.Geissendoerfer@Teradata.com

03/03/2026

© Teradata. All rights reserved. However, you may combine your original work with publicly-available, Teradata-owned documentation to make a Custom Document; provided you limit access to the Custom Document to you (and, if you are acting on behalf of a company, to that company for that company's internal use). TERADATA DISCLAIMS ALL REPRESENTATIONS, CONDITIONS, AND WARRANTIES OF ANY KIND (EXPRESS, IMPLIED, OR STATUTORY) WITH RESPECT TO THIS CUSTOM DOCUMENT. YOU AND PERSONS ACCESSING THIS CUSTOM DOCUMENT BEAR THE ENTIRE RISK OF ANY RELIANCE ON THIS CUSTOM DOCUMENT, INCLUDING AS TO QUALITY, ACCURACY, AND RESULTS.

Introduction to the Enterprise Vector Store User Guide

Using the Teradata Vector Store User Guide

The Teradata Vector Store User Guide provides an overview and details about Teradata Vector Store, the supported features, and how to use this product.

The guide also includes information about using Vector store with Python and REST APIs.

For Vector store on AI Factory, see Teradata® AI Factory and for VantageCloud Lake, see Get Started with VantageCloud Lake.

Why Would I Use this Content?

This content explains how to work with the Teradata Vector store. You can get acquainted with:

- Vector Store concepts
- Supported features and editions
- Setting up vector store
- Creating different types of vector store
- Managing Vector stores

How Do I Use this Content?

To understand the fundamentals of vector store, start with Teradata Enterprise Vector Store Fundamentals and get familiar with the product. Then you can go through Teradata Vector Store Supported Features to get an understanding of what capabilities are supported followed by Teradata Vector Store Concepts to understand more details on the concepts and features.

© 2020 Teradata. All rights reserved.

If you are an administrator, refer to Administering Vector Store and Setting up Vector Store to understand administration, configurations, and setup details.

As a user, you can create vector store in different ways. See Creating a Vector Store. Once created, you can manage it as explained in Managing a Vector Store.

Teradata Enterprise Vector Store Fundamentals

What is a vector store?

Teradata Enterprise Vector Store is a specialized database designed to store and manage high-dimensional *vectors*, that is, numerical representations of data like text, images, or user behavior. Rather than rely on exact keyword matches, vector stores use a semantic search by storing data as vectors.

You can use vector store with easy to use APIs to build your own natural language (NL) applications using Vantage as the foundational compute/storage engine.

You can use vector store in the following ways:

- Python via teradatagenai package - a Generative AI package developed by Teradata (available only in VantageCloud Lake)
- REST APIs (available in VantageCloud Enterprise and AI Factory)
- User Interface (available only in VantageCloud Lake)

What are the main uses of vector store?

You can use Enterprise Vector Store to perform the following operations:

- Semantic search and retrieval: Search content based on meaning, not just keywords. For example, if you ask a question in natural language, the system can pull relevant documents even if the documents don't contain the exact words in the query.
- AI-assisted content discovery: Generate FAQ or Q&A documents and knowledge-based enrichment content.
- Scale the search: Search millions of vectors in large content repositories.
- Manage semantic search: Manage semantic search, such as, create, edit, delete, and run administrative tasks with vector stores.

The following are some key points about Teradata Enterprise Vector Store:

- Vector data storage: You can store large datasets of vector representations (arrays) derived from the text or images using an appropriate model. You can also compare them based on semantic search functionalities.
- Scalable for large datasets: You can scale to large datasets with efficient indexing and retrieval.
- Integration with AI workflows: You can integrate your machine learning models with the vector store and perform similarity searches, generate recommendations, or analyze relationships within complex data.
- Enterprise-grade features: Teradata Vector store offers enterprise grade features for security, governance, and reliability.

Vector store is available for VantageCloud Lake, VantageCloud Enterprise, and VantageCore IntelliFlex. Contact your account team for more details.

Teradata Vector Store Supported Features

The following are the Vector store supported features that you can use:

Features	Description	Supported
Vector store functionality	Support of create(), update(), delete(), ask() APIs for: • Content-based vector store • Metadata-based vector store • File-based vector store • Embedding-based vector store	Yes
Search algorithms (TD_VectorDistance, TD_KMeans, TD_HNSW)	Provide support of three algorithms for indexing and search retrieval.	Yes
Update support for indexing and search	Update of vector store index with new data (manually triggered using update() API).	Yes
Authorization support	Authorize users to manage and use vector store collections	Yes
RAG workflow	Support document ingest, vectorization and retrieval.	Yes
Embedding generation	AI_TextEmbedding function for embedding generation	Yes
Vector Data Type	Incorporation of new 64-bit float data type	Yes
REST APIs	REST APIs for Vector store	Yes
teradatagenai (Python) APIs	Python package for Vector store APIs	Yes
Unstructured data	PDF ingest support	Yes
Basic chunking for pdf parsing	Basic chunking support based on a fixed chunk size	Yes
Optimized chunking for pdf parsing	Optimized chunking support (dynamic parsing based on sections / paragraphs identification) using nlm-ingestor.	Yes
Rank fusion	Rank fusion based reranking using BM25 on search results.	Yes
OTF support	Support of input data in OTF.	Yes
AWS cloud support with Bedrock	Support of vector store on AWS VCL with embedding and chat models from AWS bedrock.	Yes

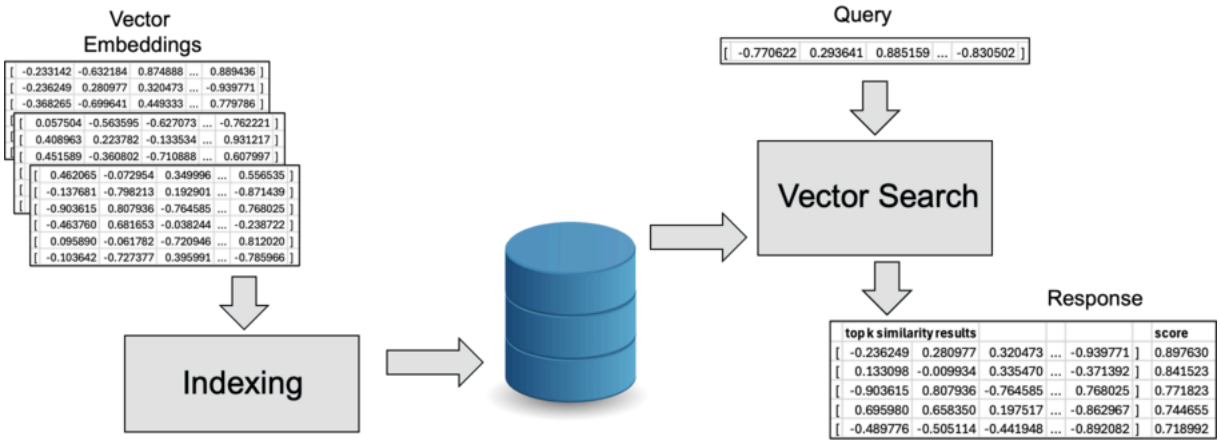
Features	Description	Supported
Azure cloud support with Azure OpenAI	Support of vector store on Azure VCL with embedding and chat models from Azure OpenAI.	Yes
ASK Batch support	Batch support for ASK APIs.	Yes
Asynchronous API support	Asynchronous API support for vector store creation, update and delete.	Yes
Concurrency support	Concurrency support of 40 users for Vector Store APIs.	Yes
Complex structure table support	Complex structure table support for graph based retrieval in HNSW.	Yes
Nvidia nv-ingest integration	Integration of Nvidia nv-ingest (tables/images/charts) for pdf parsing.	Yes
Nvidia embedding integration	Integration of Nvidia embedding LLM for embedding generation	Yes
Nvidia chat LLM integration	Integration of Nvidia chat LLM for RAG workflow.	Yes
Metadata vector store	Metadata based vector store	Yes
Pattern support for Metadata	Pattern support for Metadata based vector store.	Yes
Complex structure table support	Support of groups in complex structure table to enable backup and restore.	Yes
BYO Embeddings (Vector Ingest support)	Support of pre-embedded input data in tables.	Yes
PDF based Vector store updates	Support for addition and deletion of pdf files from a vector store.	Yes

See Supported Features Details for more details on the features.

Teradata Vector Store Concepts

Vector Store Architecture

The basic architecture of vector store is:



The following are the three main steps in generating a vector store.

- Embedding
- Indexing
- Search

Embedding Generation

The vector embeddings are generated using an embedding model, typically based on a transformer.

Embedding Models

Here are the supported embedding models for different environments.

Platform	Model Name	Default	Embedding Dimensions	Input Token Limit
AWS	amazon.titan-embed-text-v1	No	1536	8191
AWS	amazon.titan-embed-image-v1	No	1024, (384, 356)	256
AWS	amazon.titan-embed-text-v2:0	Yes	1024, (512, 256)	8191

Platform	Model Name	Default	Embedding Dimensions	Input Token Limit
Azure	text-embedding-ada-002	No	1536	8191
Azure	text-embedding-3-small	Yes	1536 (1-1536)	8191
Azure	text-embedding-3-large	No	3072 (1-3072)	8191
VantageCore	Nvidia NIM Embedding models	No	Variable	Variable

Chat Completion Models

Here are the supported Chat completion models for different environments.

Platform	Model Name	Max Tokens	Max Output Tokens
AWS	anthropic.claude-3-haiku-20240307-v1:0	200k	4096
AWS	anthropic.claude-instant-v1	100k	4096
AWS	anthropic.claude-3-5-sonnet-20240620-v1:0	200k	8192
Azure	gpt-35-turbo-16k	16384	4096
Azure	gpt-4.1-mini	1047576	32768
VantageCore	Nvidia NIM retriever and chat models	Variable	Variable

Indexing

The indexing organizes the data to facilitate faster search and is done after embeddings are generated and completes the vector store.

Search

You can query a vector store for any data and retrieve the top k matching results based on semantic similarity of a question against a vector store. For a content-based vector store, it provides a list of relevant chunks / rows and for a metadata-based vector store, it provides a list of relevant tables and columns.

Teradata provides these indexing and search algorithms:

- VectorDistance - TD_VectorDistance computes similarity or dissimilarity between two vectors in multi-dimensional space. See TD_VectorDistance.
- KMeans - The k-means algorithm groups a set of observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid). See TD_KMeans.
- HNSW - The Hierarchical Navigable Small World (HNSW) is a graph-based algorithm that performs approximate nearest neighbor searches in vector databases. See TD_HNSW.

These algorithms support the following distance metrics for similarity computation:

- Euclidean: Computes the similarity using euclidean distance between the vectors.
- Cosine: Computes the cosine similarity (normalized dot product) between two vectors.
- Dot Product: Computes the similarity between vectors using dot product.

The top k results from search are sent with a prompt to an inference LLM to generate a text response for the question.

These are the supported inference models:

Platform	Model Name	Max Tokens	Max Output Tokens
AWS	anthropic.claude-3-haiku-20240307-v1:0	200k	4096
AWS	anthropic.claude-instant-v1	100k	4096
AWS	anthropic.claude-3-5-sonnet-20240620-v1:0	200k	8192
Azure	gpt-35-turbo-16k	16384	4096
Azure	gpt-4.1-mini	1047576	32768
VantageCore	Nvidia NIM retriever and chat models	Variable	Variable

Vector Store Types

You can create a vector store using these types:

- Content-based vector store: You can build a vector store using the data from a table, file, or a PDF. Based on the question you ask against the table contents, top matches of relevant rows are returned. You can follow this with a textual response generated using an LLM by modifying the top matches.
- Metadata-based vector store: You can build a vector store using the metadata of a set of tables. Based on the question you ask against a table or set of tables, the top table matches are returned. You can have these top matches with an SQL generation component to generate an SQL query to retrieve data from relevant tables and prepare a textual response.
  Metadata-based vector store is not supported from the Vector Store User Interface on VantageCloud Lake.
- File-based vector store: You can create a file-based vector store by passing files or directory paths to the document_files parameter. You can also pass the path as wildcards. Currently only PDF files are supported as input.

- Embedding-based vector store: You can create an embedding-based vector store by supplying the user-generated embedded data to object_names. You can create this vector store by setting the is_embedded parameter to True.

Vector Store Components and Features

Enterprise Vector Store includes the following components and features:

- Database Engine 20
- VECTOR Data Type
- In-Database Analytic Functions for Vector Store
- Supported Features Details
- Schema for Content-based Vector Store
- Schema for Metadata-based Vector Store

Database Engine 20

Database Engine 20

Vector store uses the Teradata database engine 20.00 to store the vector data. It uses a special 64-bit float data type named VECTOR, to store embeddings. The DB Engine 20 also supports SLES 15, in-place upgrades, and many other features. See Database Introduction for more details.

VECTOR Data Type

Vector store uses a special UDT-based data type, VECTOR, to store vectors. Vectors are represented by arrays of float values representing embeddings. The size of the float array is controlled by embedding dimensions supported by an embedding LLM.

A VECTOR is a UDT that supports two constructors to form vectors - VARCHAR and VARBYTE. Internally, vectors are represented by varbyte (64000), which is a packed representation of embeddings. A maximum of 4096 embedding dimensions are currently supported.

Convert Data to VECTOR from VARCHAR or VARBYTE

Here are some ways to transform data into vectors. You can use them as required.

Insert records in a table - one row at a time

Python

In Python, create a pandas DataFrame with float string values or varbyte values. In this example, 1.2 has a 64-bit float representation of 333333333333F33F, 1.5 is equivalent to 000000000000F83F, and 1.8 is represented by CDCCCCCCCCCCFC3F.

```
# Creating the pandas DataFrame
data = {
    "ID": [7, 8, 9],  # Integer column
    "Values": ["-0.06944,0.080352,0.045963,0.006985,-0.000496", '333333333333F33F000000000000F83FCDCCCCCCCCCCFC3F'XB, "-0.025461,0.045302
}
df = pd.DataFrame(data)
```

Insert the data inside the database from pandas DataFrame.

```
from teradataml import copy_to_sql, DataFrame
copy_to_sql(df=df, table_name="test_float", if_exists="replace", types={"Values": VECTOR})
data = DataFrame('test_float')
```

SQL

Create a table TestTable with column embedding of VECTOR UDT.

```
CREATE MULTISET TABLE TestTable(
    id INTEGER,
    embedding Vector) PRIMARY INDEX (id);
```

Insert VARCHAR data into the VECTOR UDT.

```
INSERT INTO TestTable VALUES(1, NEW Vector('-0.06944,0.080352,0.045963,0.006985,-0.000496'));
INSERT INTO TestTable VALUES(2, '-0.025461,0.045302,-0.028107,0.031248,0.00077');
```

Insert VARBYTE data into the VECTOR UDT. For varbyte, data is represented as a sequence of 8-byte double precision values.

For example, the following two statements are equivalent.

```
INSERT INTO TestTable VALUES(7,'F4C308E1D1C6B1BFA20E2BDCF291B43FC3EFA65B7688A73FEA3E00A94D9C7C3F213EB0E3BF4040BF'XB);
INSERT INTO TestTable VALUES(7,'-0.06944,0.080352,0.045963,0.006985,-0.000496');
```

In this example, -0.06944 has a 64-bit float representation of F4C308E1D1C6B1BF, 0.080352 is equivalent to A20E2BDCF291B43F, and so on.

Insert VARCHAR Data in a Table from a File

The following example shows inserting data from a CSV file into the VECTOR column, embeddingcol, of data table, TestTable.

CSV File

```
1|-0.06944,0.080352,0.045963,0.006985,-0.000496,
2|-0.025461,0.045302,-0.028107,0.031248,0.00077,
3|-0.028107,0.016781,-0.023147,-0.068779,-0.07936
```

Load the data from CSV to table using Python or SQL.

Python

```
from teradataml import read_csv, DataFrame
# connect to vanatage
create_context(host='host', user='user', password='pass')
# Set the datatype from columns of table
types = OrderedDict(id=BIGINT, array_col=VECTOR)
# Read csv from local and load data into table inside the database.
rcsv_hnsw = read_csv(filepath='testcsv.csv',
                     table_name="my_vector",
                     types=types)
# If data is huge, use fastload to load data into database.
rcsv_hnsw = read_csv(filepath='testcsv.csv',
                     table_name="my_vector",
                     types=types,
                     use_fastload=True)
# Dataframe of table
df = DataFrame("my_vector")
```

SQL - Example 1

Create a table with VARCHAR column.

```
DROP TABLE TestTable;
CREATE MULTISET TABLE TestTable(
    id INTEGER,
    embeddingcol Varchar(64000)) NO PRIMARY INDEX;
```

Load using tdload.

```
tdload -h localhost -u alice -p alice -f test.csv -d '|' -t TestTable myjob1
```

Insert into the VECTOR column.

```
DROP TABLE TestTable_1;
CREATE MULTISET TABLE TestTable_1(
    id INTEGER,
    embeddingcol Vector) NO PRIMARY INDEX;
Insert into TestTable_1 Select * from TestTable;
```

SQL - Example 2

Create a table in Vector.

```
DROP TABLE TestTable;
CREATE MULTISET TABLE TestTable(
   id INTEGER,
   embeddingcol Vector) NO PRIMARY INDEX;
```

Import csv file and insert into table.

```
.IMPORT VARTEXT FILE = ./testcsv.csv
.SET PACK 20
.REPEAT *
USING (id varchar(100), embeddingcol VARCHAR(100))
INSERT INTO TestTable
VALUES (:id, :embeddingcol);
```

Insert VARBYTE Data in a Table - from a File

You can use the following example to load a text file containing VARBYTE data into a table with a VECTOR column.

testcsv_varbyte.csv

```
1|F4C308E1D1C6B1BFA20E2BDCF291B43FC3EFA65B7688A73FEA3E00A94D9C7C3F213EB0E3BF4040BF
3|A29927D714C89CBF13D6C6D8092F913F388600E0D8B397BF36CEA623809BB1BFA94D9CDCEF50B4BF
2|0D384BC972129ABF4704E3E0D231A73FA29927D714C89CBF51FA42C879FF9F3F419AB1683A3B493F
```

Python

```
from teradataml import read_csv, DataFrame
# connect to vanatage
create_context(host='host', user='user', password='pass')
# Set the datatype from columns of table
types = OrderedDict(id=BIGINT, array_col=VECTOR)
# Read csv from local and load data into table inside the database.
rcsv_hnsw = read_csv(filepath='testcsv_varbyte.csv',
                     table_name="my_vector",
                     types=types)
# Dataframe of table
df = DataFrame("my_vector")
```

SQL - Example 1

```
DROP TABLE TestTable;
CREATE MULTISET TABLE TestTable(
   id INTEGER,
   embeddingcol varbyte(100)) NO PRIMARY INDEX;
```

Create a table in VARBYTE.

Load using tdload

```
tdload -h localhost -u alice -p alice -f testcsv_varbyte.txt -d '|' -t TestTable myjob2
```

Insert into VECTOR column.

```
DROP TABLE TestTable_1;
CREATE MULTISET TABLE TestTable_1(
   id INTEGER,
   embeddingcol Vector) NO PRIMARY INDEX;

Insert into TestTable_1 Select id, to_bytes(embeddingcol, 'base16') from TestTable;
```

SQL - Example 2

```
DROP TABLE TestTable_1;
CREATE MULTISET TABLE TestTable_1(
   id INTEGER,
   embeddingcol Vector) NO PRIMARY INDEX;
Insert into TestTable_1 Select id, to_bytes(embeddingcol, 'base16') from TestTable;
```

```
     .IMPORT VARTEXT FILE = ./testcsv_varbyte.csv
     .SET PACK 20
     .REPEAT *
     USING (id varchar(100), embeddingcol VARCHAR(100))
     INSERT INTO TestTable
     VALUES (:id, to_bytes(:embeddingcol, 'base16'));
```

Copying from One Table to Another

You can copy data into a vector column from another column containing varchar or varbyte data.

Python

In Python, you can copy data from one table to another using copy_to_sql().

```
from teradataml import DataFrame, copy_to_sql, create_context
# connect to vantage
create_context(host='host', user='user', password='pass')
# create a table 'InputTable' containing Vector Data.
# Create a teradataml dataframe on 'InputTable' table
tab_to_copy = DataFrame('InputTable')
# copy 'InputTable' into another table 'test_table' in the database
copy_to_sql(df=tab_to_copy, table_name='test_table', if_exists='replace', types={text_data:VECTOR})
```

SQL

```
CREATE MULTISET TABLE InputTable(
    id INTEGER,
    text_data varchar(100)) PRIMARY INDEX (id);
INSERT INTO InputTable VALUES(1, '-0.06944,0.080352,0.045963,0.006985,-0.000496');
INSERT INTO InputTable VALUES(2, '-0.025461,0.045302,-0.028107,0.031248,0.00077');
INSERT INTO InputTable VALUES(3, '-0.028107,0.016781,-0.023147,-0.068779,-0.07936');
CREATE MULTISET TABLE TestTable(
```

```
    id INTEGER,
    embedding Vector) PRIMARY INDEX (id);
INSERT INTO TestTable SELECT * FROM InputTable;
```

Convert to VARCHAR

Python

In Python, you can use cast() to convert byte data into varchar.

```
df = DataFrame('TestTable')
df.embedding.cast(varchar(100))
```

SQL

While vector contents are stored in byte format, the byte data can be converted to varchar using CAST.

```
CREATE MULTISET TABLE TestTable(
    id INTEGER,
    embedding Vector) PRIMARY INDEX (id);
INSERT INTO TestTable VALUES(1, '-0.06944,0.080352,0.045963,0.006985,-0.000496');
INSERT INTO TestTable VALUES(2, '-0.025461,0.045302,-0.028107,0.031248,0.00077');
INSERT INTO TestTable VALUES(3, '-0.028107,0.016781,-0.023147,-0.068779,-0.07936');
SELECT id, CAST(embedding as varchar(100)) as embedding from TestTable;
```

Checking the Dimension of the Vector Data

The check the dimension of the embedding data, you can use .length() on the Vector column.

Python

Assume you have a table TestTable with embedding column containing data with Vector Datatype.

To get the length of the embedding, use the following code.

```
from teradataml.dataframe.sql import _SQLColumnExpression
col = _SQLColumnExpression('embedding.length()', type=INTEGER())
df.assign(n=col.distinct(), drop_columns=True)
```

SQL

```
CREATE MULTISET TABLE TestTable(
    id INTEGER,
    embedding Vector) PRIMARY INDEX (id);
INSERT INTO TestTable VALUES(1, '-0.06944,0.080352,0.045963,0.006985,-0.000496');
INSERT INTO TestTable VALUES(2, '-0.025461,0.045302,-0.028107,0.031248,0.00077');
INSERT INTO TestTable VALUES(3, '-0.028107,0.016781,-0.023147,-0.068779,-0.07936');
SELECT embedding.length() from TestTable;
 *** Query completed. 2 rows found. One column returned.
 *** Total elapsed time was 1 second.
embedding.LENGTH()
------------------
                 5
                 5
                 5
SELECT DISTINCT(embedding.length()) from TestTable;
 *** Query completed. One row found. One column returned.
 *** Total elapsed time was 1 second.
embedding.LENGTH()
------------------
                 5
```

In-Database Analytic Functions for Vector Store

Vector store uses Database Engine 20 functions to embed, index, and search in a large data. The following are the functions you can use for vector store.

AI_TextEmbeddings and Vector Store

AI_TextEmbedding generates embeddings of input data stored in a VARCHAR column in a database table. It connects to the embedding model served by AWS Bedrock or Azure OpenAI and sends requests from input table for embedding generation.

For AWS Bedrock, the function supports batch mode by packing multiple requests in a single batch for better throughput. A batch request can have a minimum of 50 requests (rows) and a maximum of 50K. The function outputs a table containing embeddings, error messages in case of failed attempts , and token counts per request.

Embedding information is returned in the column name embedding, error messages are returned as the column message, and token information is returned in the column called num_tokens.

Important: You cannot have columns named embedding, message, or num_tokens in your table. To avoid errors, rename those columns before using AI_TextEmbedding.

You can use the following example to create an embedding generation on Bedrock (equivalent to Python).

Python

```
create authorization td_gen_azure_auth
       user '{API_BASE}'
       password '{API_KEY}'
       Session_Token '{API_VERSION}';
```

```
# Importing required packages
import sys
import getpass, os
from teradatagenai import TeradataAI, TextAnalyticsAI, load_data
from teradataml import create_context, set_config_params, list_base_envs, list_user_envs, create_env
# Connecting to vantage using 'create_context()' from teradataml
user = getpass.getpass(prompt='Enter username: ')
password = getpass.getpass(prompt='Enter password: ')
host = getpass.getpass(prompt='Enter host: ')
eng = create_context(host=host, username=user, password=password)
load_data('employee', 'employee_data')
df=DataFrame('employee_data')
df_articles = df.select(["employee_id", "employee_name", "articles"])
llm_azure = TeradataAI(api_type="azure",
                       authorization="td_gen_azure_auth",
                       model_name="{MODEL_NAME}")
ta_obj = TextAnalyticsAI(llm=llm_azure)
ta_obj.embeddings("articles",df_articles)
```

SQL

You can generate an embedding the VARBYTE format using Azure OpenAI model text-embeddings-3-small.

```
SELECT * FROM AI_TextEmbeddings(
      ON ta_employee_data AS InputTable
      USING
      TextColumn('articles')
      ApiType('azure')
      Authorization(td_gen_azure_auth)
      DeploymentId('<engine id>')
      ModelName('text-embeddings-3-small')
      isDebug('true')
      Accumulate('[0:]')
      OutputFormat('varbyte')
    ) as dt;
```

The following example uses Bedrock model amazon.titan-embed-text-v1 to generate embeddings in vector UDT format.

```
SELECT * FROM AI_TextEmbeddings(
      ON ta_employee_data AS InputTable
      USING
      TextColumn('articles')
      ApiType('aws')
      REGION('us-west-2')
      ACCESSKEY('<access key>')
      SECRETKEY('<secret key>')
      ModelName('amazon.titan-embed-text-v1')
      outputformat('varbyte')
      isDebug('true')
      Accumulate('[0:]')
    ) as dt;
```

The following example uses Bedrock model amazon.titan-embed-text-v1 to generate embeddings in varbyte format (Batch mode).

```
SELECT Embedding, Message FROM AI_TEXTEMBEDDINGS(
      ON (SELECT dt.review, td_byone() FROM food_reviews dt) AS InputTable PARTITION BY TD_BYONE()
      USING authorization(alice.testawsauth)
      TextColumn('review')
      ApiType('aws')
      REGION('us-west-2')
      ModelName('amazon.titan-embed-text-v1')
      outputformat('varbyte')
      batchargs('{
    "inputDataConfig": {
```

```
        "s3InputDataConfig": {
            "s3Uri": "s3://td-amzn-s3-demo-bucket-input/test-batch.jsonl"
        }
    },
    "jobName": "test-batch-emb",
    "modelId": "amazon.titan-embed-text-v2:0",
    "outputDataConfig": {
        "s3OutputDataConfig": {
            "s3Uri": "s3://td-amzn-s3-demo-bucket-output/"
        }
    },
    "roleArn": "arn:aws:iam::076782961461:role/td-bedrock-aws-batch"
}')
      isDebug('true')
      Accumulate('[0:]')
    ) as dt;
```

TD_VectorDistance and Vector Store

TD_VectorDistance computes similarity or dissimilarity between two vectors in multi-dimensional space. The distance between vectors is usually calculated using a distance metric, such as Euclidean, Manhattan, or Cosine. It takes a table of target vectors, and a table of reference vectors and returns a table that contains the distance between target-reference pairs. The function computes the distance between the target pair and the reference pair from the same table if you provide only one table as the input.

TD_VectorDistance provides flat index capability and does not require any training of the index. It performs search in a scalable and massively parallel environment and the execution time grows linearly with the input data size. Because of this, it is only suitable for smaller datasets (for example., <10K documents) to keep response time latency within an acceptable human perception (for example, <500 milliseconds).

Example

Given a vector store table “vectorstore_ms_marco_mrr_index” with id column rev_id and embeddings column vector_index, the following query finds top k results for the query specified in the TargetTable.

```
SELECT reference_id, similarity FROM TD_VECTORDISTANCE(
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS TargetTable DI
  ON vectorstore_ms_marco_mrr_index AS ReferenceTable
  USING
  TargetIDColumn('queryID')
  TargetFeatureColumns('queryEmbedding')
  RefIDColumn('TD_TEMP_ID')
  RefFeatureColumns('vector_index')
  DistanceMeasure('EUCLIDEAN')
  OutputSimilarity('t')
  TopK(10)
  LargeReferenceInput('t')
  EmbeddingSize(1024)
) AS dt ORDER BY similarity DESC;
```

See TD_VectorDistance for more details.

TD_KMeans and Vector Store

The k-means algorithm groups a set of observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid).

The index training runs in a parallel distributed environment and produces two tables; a centroids table (model table) containing the vectors representing cluster centers (centroids) and a vector table containing all data points associated to the closest centroid.

During search, the centroids table is scanned first to narrow down the search to top n cluster matches. Then search is performed against all data points belonging to these top clusters to find top k matches for similarity search. KMeans is an approximate approach; the search quality can be controlled by adjusting the value of n in the first scan of cluster centroids at the expense of search latency. TD_KMeans scales to very large datasets and is a preferable approach for large scales while keeping latency within acceptable limits.

Example

Here is an example usage of the function. Given a vector store table “vectorstore_ms_marco_mrr_index” with id column ‘rev_id’ and embeddings column ‘vector_index’, following query is run to train the index.

```
-- kmeans training
CREATE MULTISET TABLE vectorstore_ms_marco_mrr_kmeans_model as (
SELECT * FROM TD_KMEANS (
  ON vectorstore_ms_marco_mrr_index AS InputTable
  USING
  IdColumn('TD_TEMP_ID')
  TargetColumns('vector_index')
  InitialCentroidsMethod('RANDOM')
  NumClusters(700)
  Seed(0)
```

```
  StopThreshold(0.0395)
  MaxIterNum(10)
  NumInit(1)
  EmbeddingSize(1024)
) AS dt) WITH DATA NO PRIMARY INDEX;

-- update index table with cluster assignments
CREATE MULTISET TABLE vectorstore_ms_marco_mrr_index AS (
SELECT pid, rev_id, TD_TEMP_ID, vector_index, vector_index_normalized, td_clusterid_kmeans as clusterID FROM TD_KMEANSPREDICT(
  ON vectorstore_ms_marco_mrr_index_old AS InputTable
  ON vectorstore_ms_marco_mrr_kmeans_model AS ModelTable DIMENSION
  USING
  Accumulate('pid','rev_id','vector_index','vector_index_normalized')
) AS dt) WITH DATA PRIMARY INDEX (clusterID);

-- create centroids view
REPLACE VIEW vectorstore_ms_marco_mrr_centroids AS (
  SELECT td_clusterid_kmeans as clusterID, vector_index AS centroid FROM vectorstore_ms_marco_mrr_kmeans_model
    WHERE td_clusterid_kmeans IS NOT Null);
```

You can use the following query to find top k results for the query specified in the TargetTable.

```
--search in the centroid table first to get the top n clusters
SELECT reference_id FROM TD_VECTORDISTANCE(
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS TargetTable DI
  ON vectorstore_ms_marco_mrr_centroids AS ReferenceTable
  USING
  TargetIDColumn('queryID')
  TargetFeatureColumns('queryEmbedding')
  RefIDColumn('clusterID')
  RefFeatureColumns('centroid')
  DistanceMeasure('Euclidean')
  OutputSimilarity('t')
  Topk(100)
  LargeReferenceInput('t')
  EmbeddingSize(1024)
) AS dt ORDER BY similarity DESC;

--search in the vector store to get the top k results
SELECT reference_id, similarity FROM TD_VECTORDISTANCE(
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS TargetTable DI
  ON (SELECT * FROM vectorstore_ms_marco_mrr_index WHERE clusterID in (1,0,2,...) ) AS ReferenceTable
  USING
  TargetIDColumn('queryID')
  TargetFeatureColumns('queryEmbedding')
  RefIDColumn('TD_TEMP_ID')
  RefFeatureColumns('vector_index')
  DistanceMeasure('EUCLIDEAN')
  OutputSimilarity('t')
  TopK(10)
  LargeReferenceInput('t')
  EmbeddingSize(1024)
) AS dt ORDER BY similarity DESC;
```

See TD_KMeans for more details.for details and examples.

TD_HNSW and Vector Store

The Hierarchical Navigable Small World (HNSW) is a graph-based algorithm that performs approximate nearest neighbor searches in vector databases.

It uses a multi-layered graph structure to efficiently search high-dimensional spaces. The top layer has a sparse graph with long-range connections, and the lower layers become denser. The search starts at the top layer, then moves down through the layers to find the nearest neighbors.

The index training runs in a parallel distributed environment and builds a nearest neighbor layered graph using a framework optimized for graph navigation called complex structure table (CST). The layers are constructed in such a way that top most layer contains the fewest number of nodes and graph gets denser as it goes down the layer.

The search starts from the top most layer, finds the nearest neighbor at that layer, which becomes the starting point of the next layer. The bottom layer contains all data points as nodes and search in that layer returns the final top k matches for similarity search. Thus, each layer plays a role to narrow down the search results. HNSW is an approximate search algorithm and scales to very large datasets while maintain good quality of results.

Example

Here is an example usage of the function.

Given a vector store table “vectorstore_ms_marco_mrr_index” with id column ‘rev_id’ and embeddings column ‘vector_index’, following query is run to train the index.

```
--hnsw training
SELECT * FROM TD_HNSW (
  ON vectorstore_ms_marco_mrr_index AS InputTable
  OUT PERMANENT TABLE ModelTable(vectorstore_ms_marco_mrr_hnsw_model)
  USING
  IdColumn('TD_TEMP_ID')
  VectorColumn('vector_index')
  Seed(0)
  EfConstruction(32)
  NumConnPerNode(32)
  MaxNumConnPerNode(32)
  DistanceMeasure('EUCLIDEAN')
  EmbeddingSize(1024)
  ApplyHeuristics('True')
) as dt;
```

You can use the following query to find top k results for the query specified in the TargetTable.

```
--hnsw search
SELECT nearest_neighbor_id, similarity FROM TD_HNSWPredict(
  ON vectorstore_ms_marco_mrr_hnsw_model AS ModelTable
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS InputTable DIM
  USING
  IdColumn('queryID')
  VectorColumn('queryEmbedding')
  EfSearch(32)
  OutputSimilarity('t')
  SingleInputRow('t')
  TopK(10)
) AS dt ORDER BY similarity DESC;
```

See TD_HNSW for more details.

AI_AskLLM

The AI_AskLLM generates a textual response based on an input question with the help of a prompt and context. It connects to the inference model served by AWS Bedrock or Azure OpenAI and sends requests from input table for response generation. Multiple questions can be packed together in the input table along with the corresponding context. The function outputs a table containing response and error message in case of failed attempt.

Python

Example usage of the function in Azure.

```
create authorization td_gen_azure_auth
       user '{API_BASE}'
       password '{API_KEY}'
       Session_Token '{API_VERSION}';
```

```
# Importing required packages
import sys
import getpass, os
from teradatagenai import TeradataAI, TextAnalyticsAI, load_data
from teradataml import create_context, set_config_params, list_base_envs, list_user_envs, create_env
# Connecting to vantage using 'create_context()' from teradataml
user = getpass.getpass(prompt='Enter username: ')
password = getpass.getpass(prompt='Enter password: ')
host = getpass.getpass(prompt='Enter host: ')
eng = create_context(host=host, username=user, password=password)
load_data('employee', 'context_data')
load_data('employee', 'questions')
context=DataFrame('context_data')
question=DataFrame('questions')
llm_azure = TeradataAI(api_type="azure",
                       authorization="td_gen_azure_auth",
                       model_name="{MODEL_NAME}")
ta_obj = TextAnalyticsAI(llm=llm_azure)
ta_obj.Ask(data=question,
                context= context ,
                text_column='text_data',
                context_column='text_data',
                data_partition_column='id',
                context_partition_column='id',
                prompt='Provide an answer to the question using data as information relevant to the question. \nQuestion: #QUESTION# \n D
                data_position='#DATA#',
```

```
                question_position='#QUESTION#'
                )
```

SQL

Example usage of function in Azure.

```
SELECT * FROM AI_AskLLM(
      ON question AS InputTable partition by id
      ON context_data AS ContextTable partition by id
      USING
      TextColumn('text_data')
      ApiType('azure')
      ApiBase('<api base>')
      ApiVersion('<api version>')
      ApiKey('<api key>')
      DeploymentId('<engine id>')
      ModelName('gpt-3.5-turbo')
      ContextColumn('text_data')
      Prompt('Provide an answer to the question using data as information relevant to the question. \nQuestion: #QUESTION# \n Data: #DATA
      DATAPOSITION('#DATA#')
      QUESTIONPOSITION('#QUESTION#')
      isDebug('true')
      Accumulate('[0:]')
    ) as dt;
```

Example usage of the function on AWS.

```
SELECT * FROM AI_AskLLM(
      ON question AS InputTable partition by id
      ON context_data AS ContextTable partition by id
      USING
      TextColumn('text_data')
      ApiType('aws')
      REGION('us-west-2')
      ACCESSKEY('<access key>')
      SECRETKEY('<secret key>')
      ModelName('anthropic.claude-v2')
      ContextColumn('text_data')
      Prompt('Provide an answer to the question using data as information relevant to the question. \nQuestion: #QUESTION# \n Data: #DATA
      DATAPOSITION('#DATA#')
      QUESTIONPOSITION('#QUESTION#')
      isDebug('true')
      Accumulate('[0:]')
    ) as dt;
```

Supported Features Details

The following sections provide more details of the Vector store supported features.

Ingest Support

Vector store ingests structured and unstructured data in different ways.

Structured Data

For content-based vector store, input data can be specified using columns of database tables and views. These columns contain text data that needs to be searched for similarity. Some examples of such data include articles, abstracts, product reviews, customer complaints, chunked documents etc. The specified columns are vectorized using embedding models and integrated into the vector store.

Content-based vector stores also support pre-embedded data. You can use your own embeddings, store them into tables and create vector store with them.

For metadata-based vector store, input data is specified in the form of database tables and views. The metadata of these tables (column names, data types, foreign keys, join, and so on) is used for embedding and integrated into the vector store. Some examples of such data includes employees and department information, product sales, transactions, bank accounting, etc. and the search query is to find top matches of tables and columns containing data. These tables and columns can then be used for SQL generation to retrieve the output from these tables.

Unstructured Data

Content-based vector stores can take input data in the form of PDFs. A pdf ingest pipeline ingests files, chunks and integrates into the vector store. Questions can be asked against the contents of the pdf to obtain relevant matches. Currently, only text data within PDFs is vectorized.

There are two chunking options available:

- Fixed-size chunking: The pdf document file is split into chunks of fixed length specified by chunk_size.

- Optimized chunking: The pdf document file is parsed internally based on document structure and chunks are dynamically created based on document layout. The chunk_size field is not applicable when optimized_chunking is set to True.

For both approaches, you can provide additional configuration for header and footer trimming using header_height and footer_height parameters.

NV - Ingest

Vector store supports Nvidia's NeMo Retriever extraction. It is a scalable, performance-oriented document content and metadata extraction microservice. NeMo Retriever extraction uses specialized NVIDIA NIM microservices to find, contextualize, and extract text, tables, charts and images that you can use in downstream generative applications.

OTF Support

By default, vector store uses BFS storage for storing vectors. If required, you can store source data in OTF tables.You can provide them using the same parameters as BFS tables but in the format <datalake_name>.<database_name>.<table_name>.

For example:

```
'object_names':
    ['iceberg_hive_aws_lav.oaf.amazon_reviews_25'],
```

Lexical Search for Rank Fusion

Vector store supports ensembling of results from vector similarity search and BM25 based lexical search. You can enable this option by specifying rerank_weight > 0. Instead of retrieving top_k results from vector similarity search, relevance_top_k results are retrieved.

Once the results are retrieved, they are passed to BM25 retrieval algorithm. BM25 ranks documents based on their relevance to a given query using lexical keyword search. It calculates a score for each document based on presence, frequency and rarity of query terms. The value of rerank_weight determines the percentage weightage of lexical search scores.

Asynchronous APIs

Vector store APIs of create, update, or delete are asynchronous because these could be long running jobs. When user issues a request for them, e.g., create, the API starts a thread in the background to complete the steps and issues an accepted response to the user. The user can then use status API to check the status of the vector store.

Search Threshold

While top_k is the common approach to establish the criteria for returning the similarity results, vector store also support search threshold criteria. When search_threshold is supplied, the results with similarity score above the search threshold are returned.

Patterns

For metadata-based vector stores, the selection of tables can be huge. They can span multiple databases and it can become tedious to list them using include_objects and exclude_objects as mentioned in Ingest support. Patterns provide a way to select these tables and columns using simple regular expressions. A pattern string can be formed using SQL wildcards (% or _). Here are some examples:

- db1.account% : all tables and views in database db1 starting with name account.
- log__ : all tables and views starting with log and having two extra characters like log_a or log12 in the logged-in database at the time of vector store creation.
- data_.% : all tables and views in all databases starting with data and having one extra character like data1.t1, data2.v2, datax.train.

Batch Support

Vector store supports batch processing at two different stages:

- Embedding generation. This support is provided by parameter batch. A batch size can range from 100 rows to a maximum of 50K rows. More details of how to use the batch are supplied under AI_TextEmbedding section.
- Querying vector store. This support is provided by batch ask APIs.

  .

Concurrency Support

Vector store supports users to run their workloads in a concurrent environment. However, there are a few limitations that should be kept into consideration:

- A single session can have a single create, update or delete call in progress at any time. A different session / workload should be initiated to update another vector store.
- A single session can have a single ask() in progress at any time as it is not async operation.
- Multiple concurrent ask() calls can be made from different sessions without any performance degradation.

Schema for Content-based Vector Store

A vector store comprises an indexing table vectorstore_<vs_name>_index and a view named vectorstoreV_<vs_name> created from the indexing table and source table (InputTable).

The following sections explain about content-based Vector store schema.

Input Table Schema

For content based vector store, key columns and data columns are important in the input table apart from the other columns.

Column Name	Column Type	Description
<key_columns>	ANY	Contains the columns that identify rows of data columns.
<data_columns>	CHAR/VARCHAR	Contains the columns that contain text data for embedding and search.

Based on this, Vector Store Schema generated for different search algorithm is as follows:

VectorDistance

Indexing table (single input data column)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column. This column is used for euclidean and dot product metrics.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column. This column is used for cosine similarity metric.

Indexing table (multiple input data columns)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

VectorStore view (single input data column)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
<data_columns>	CHAR/VARCHAR	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

Vector Store View (multiple input data columns)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
AttributeValue	CHAR/VARCHAR	Contains the input data column values.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

HNSW

Indexing table (single input data column)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.

Column Name	Column Type	Description
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

Indexing table (multiple input data columns)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

HNSW Model table

Column Name	Column Type	Description
TD_HNSW_Nodes	VARBYTE	Nodes containing HNSW graph.

VectorStore view (single input data column)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
<data_columns>	CHAR/VARCHAR	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

VectorStore view (multiple input data columns)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
AttributeValue	CHAR/VARCHAR	Contains the input data column values.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

Example

Given a vector store table “vectorstore_ms_marco_mrr_index” with id column ‘rev_id’ and embeddings column ‘vector_index’, following query is run to train the index.

```
--hnsw training
SELECT * FROM TD_HNSW (
  ON vectorstore_ms_marco_mrr_index AS InputTable
  OUT PERMANENT TABLE ModelTable(vectorstore_ms_marco_mrr_hnsw_model)
  USING
  IdColumn('TD_TEMP_ID')
  VectorColumn('vector_index')
  Seed(0)
  EfConstruction(32)
  NumConnPerNode(32)
  MaxNumConnPerNode(32)
  DistanceMeasure('EUCLIDEAN')
  EmbeddingSize(1024)
  ApplyHeuristics('True')
) as dt;
```

You can use the following query to find top k results for the query specified in the TargetTable.

```
--hnsw search
SELECT nearest_neighbor_id, similarity FROM TD_HNSWPredict(
  ON vectorstore_ms_marco_mrr_hnsw_model AS ModelTable
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS InputTable DIM
  USING
  IdColumn('queryID')
  VectorColumn('queryEmbedding')
  EfSearch(32)
  OutputSimilarity('t')
  SingleInputRow('t')
  TopK(10)
) AS dt ORDER BY similarity DESC;
```

Detailed documentation for the function is available in Database Engine 20 - In-Database Analytic Functions.

KMeans

Indexing table (single input data column)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

Indexing table (multiple input data columns)

Column Name	Column Type	Description
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

Centroids view

Column Name	Column Type	Description
clusterID	BIGINT	Identifier of a cluster.
centroid	VECTOR	Centroid vector

KMeans Model table

Column Name	Column Type	Description
td_clusterid_kmeans	BIGINT	Unique identifier of cluster.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of centroid value for each feature.
td_size_kmeans	BIGINT	Number of points in the cluster.
td_withinss_kmeans	FLOAT	Within-cluster-sum-of-squares—sum of squared differences of each point from its cluster centroid.
TD_TEMP_ID	BYTEINT	Copied from input. All NULL rows.

Column Name	Column Type	Description
td_modelinfo_kmeans	VARCHAR	Info related to the model generated. The following are the info stored: Converged: True or False Number of Iterations: number of iterations performed. Number of Clusters: number of clusters produced. Total_WithinSS: Total Within Cluster Sum Of Squares. Between_SS : Between sum of squares—sum of squared distances of centroids to global mean, where squared distance of each mean to global mean is multiplied by the number of data points it represents. Method for InitialCentroids : 'Random' or 'KMeans++' or 'Externally supplied InitialCentroidsTable'.

VectorStore view (single input data column)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
<data_columns>	CHAR/VARCHAR	Same as in Input table.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

VectorStore view (multiple input data columns)

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object.
<key_columns>	ANY	Same as in Input table.
AttributeName	VARCHAR	Contains the input data column names.
AttributeValue	CHAR/VARCHAR	Contains the input data column values.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

Example

Here is an example usage of the function. Given a vector store table “vectorstore_ms_marco_mrr_index” with id column ‘rev_id’ and embeddings column ‘vector_index’, following query is run to train the index.

```
-- kmeans training
CREATE MULTISET TABLE vectorstore_ms_marco_mrr_kmeans_model as (
SELECT * FROM TD_KMEANS (
  ON vectorstore_ms_marco_mrr_index AS InputTable
  USING
  IdColumn('TD_TEMP_ID')
  TargetColumns('vector_index')
  InitialCentroidsMethod('RANDOM')
  NumClusters(700)
  Seed(0)
  StopThreshold(0.0395)
  MaxIterNum(10)
  NumInit(1)
  EmbeddingSize(1024)
) AS dt) WITH DATA NO PRIMARY INDEX;
```

```
-- update index table with cluster assignments
CREATE MULTISET TABLE vectorstore_ms_marco_mrr_index AS (
SELECT pid, rev_id, TD_TEMP_ID, vector_index, vector_index_normalized, td_clusterid_kmeans as clusterID FROM TD_KMEANSPREDICT(
  ON vectorstore_ms_marco_mrr_index_old AS InputTable
  ON vectorstore_ms_marco_mrr_kmeans_model AS ModelTable DIMENSION
  USING
  Accumulate('pid','rev_id','vector_index','vector_index_normalized')
) AS dt) WITH DATA PRIMARY INDEX (clusterID);
-- create centroids view
REPLACE VIEW vectorstore_ms_marco_mrr_centroids AS (
  SELECT td_clusterid_kmeans as clusterID, vector_index AS centroid FROM vectorstore_ms_marco_mrr_kmeans_model
    WHERE td_clusterid_kmeans IS NOT Null);
```

You can use the following query to find top k results for the query specified in the TargetTable.

```
--search in the centroid table first to get the top n clusters
SELECT reference_id FROM TD_VECTORDISTANCE(
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS TargetTable DI
  ON vectorstore_ms_marco_mrr_centroids AS ReferenceTable
  USING
  TargetIDColumn('queryID')
  TargetFeatureColumns('queryEmbedding')
  RefIDColumn('clusterID')
  RefFeatureColumns('centroid')
  DistanceMeasure('Euclidean')
  OutputSimilarity('t')
  Topk(100)
  LargeReferenceInput('t')
  EmbeddingSize(1024)
) AS dt ORDER BY similarity DESC;
--search in the vector store to get the top k results
SELECT reference_id, similarity FROM TD_VECTORDISTANCE(
  ON (SELECT 1 AS queryID, NEW Vector('-0.004343,0.025832,0.034292,-0.030457,0.041286,0.025042,...') AS queryEmbedding) AS TargetTable DI
  ON (SELECT * FROM vectorstore_ms_marco_mrr_index WHERE clusterID in (1,0,2,...) ) AS ReferenceTable
  USING
  TargetIDColumn('queryID')
  TargetFeatureColumns('queryEmbedding')
  RefIDColumn('TD_TEMP_ID')
  RefFeatureColumns('vector_index')
  DistanceMeasure('EUCLIDEAN')
  OutputSimilarity('t')
  TopK(10)
  LargeReferenceInput('t')
  EmbeddingSize(1024)
) AS dt ORDER BY similarity DESC;
```

See In-DB Analytic Functions Explorer in Database Engine 20 - In-Database Analytic Functions for more details.

Schema for Metadata-based Vector Store

The following sections explain about metadata-based Vector store schema.

Input table

Column Name	Column Type	Description
<input_column>	ANY	Input table can have any number and type of input columns.

VectorDistance

Indexing table

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

VectorStore view

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
VectorStoreDDL	VARCHAR	Contains the schema of the source object (table/view).
VectorStoreData	VARCHAR	Contains the object/column description.
JoinColumns	VARCHAR	Contains information about foreign keys on which different objects can be joined.
SampleData	VARCHAR	Contains information about sample data from the source column.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

HNSW

Indexing table

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

HNSW Model table

Column Name	Column Type	Description
TD_HNSW_Nodes	VARBYTE	Nodes containing HNSW graph.

VectorStore view

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
VectorStoreDDL	VARCHAR	Contains the schema of the source object (table/view).
VectorStoreData	VARCHAR	Contains the object/column description.
JoinColumns	VARCHAR	Contains information about foreign keys on which different objects can be joined.
SampleData	VARCHAR	Contains information about sample data from the source column.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.

KMeans

Indexing table

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

Centroids view

Column Name	Column Type	Description
clusterID	BIGINT	Identifier of a cluster.
centroid	VECTOR	Centroid vector.

KMeans Model table

Column Name	Column Type	Description
td_clusterid_kmeans	BIGINT	Unique identifier of cluster.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of centroid value for each feature.
td_size_kmeans	BIGINT	Number of points in the cluster.
td_withinss_kmeans	FLOAT	Within-cluster-sum-of-squares—sum of squared differences of each point from its cluster centroid.
TD_TEMP_ID	BYTEINT	Copied from input. All NULL rows.
td_modelinfo_kmeans	VARCHAR	Info related to the model generated. The following are the info stored: Converged: True or False Number of Iterations: number of iterations performed. Number of Clusters: number of clusters produced. Total_WithinSS: Total Within Cluster Sum Of Squares. Between_SS : Between sum of squares—sum of squared distances of centroids to global mean, where squared distance of each mean to global mean is multiplied by the number of data points it represents. Method for InitialCentroids : 'Random' or 'KMeans++' or 'Externally supplied InitialCentroidsTable'.

VectorStore view

Column Name	Column Type	Description
DataBaseName	VARCHAR	Name of source database.
TableName	VARCHAR	Name of source object (table/view).
ColumnName	VARCHAR	Name of source column. NULL for object row.
VectorStoreDDL	VARCHAR	Contains the schema of the source object (table/view).
VectorStoreData	VARCHAR	Contains the object/column description.
JoinColumns	VARCHAR	Contains information about foreign keys on which different objects can be joined.
SampleData	VARCHAR	Contains information about sample data from the source column.
TD_TEMP_ID	BIGINT	Row identifier created internally for bookkeeping.
<vector_column>	VECTOR	Contains the vectorized representation of data column.
<vector_column>_normalized	VECTOR	Contains the normalized vectorized representation of data column.
clusterID	BIGINT	Cluster identifier to which the vector belongs.

Administering Vector Store

Vector store empowers you to efficiently manage, create, and utilize vector stores using UI and APIs.

To ensure a smooth user experience, the enterprise vector store has categorized its workflows into two primary user roles: Admin and User with each vector store having its own set of admin and users.

The following sections provide more information of administering vector store.

- Administrator Role in Vector Store
- User Role in Vector Store
- Authenticating Users

Administrator Role in Vector Store

Administrators have permissions to manage creation, configuration, and maintenance of vector stores.

Required Permissions

Vector store Administrators must have the following permissions:

- TD_AIAdmin role
- CREATE ROLE and DROP ROLE with grant option
- SELECT permission on the Data Dictionary view DBC.All_RI_ChildrenV with grant option

A DBA can grant the Admin role using the following SQL:

```
GRANT TD_AIAdmin TO <user_name>;
MODIFY USER <user_name> AS DEFAULT ROLE = ALL;
GRANT CREATE ROLE, DROP ROLE TO <user_name> WITH GRANT OPTION;
GRANT SELECT ON DBC.All_RI_ChildrenV TO <user_name> WITH GRANT OPTION;
```

Once granted the TD_AIAdmin role, the user can create new vector stores. However, they need access permissions to view the existing ones.

You can't access the vector store using SSO credentials. The workaround is to grant the TD_AIAdmin role to the user with SSO credentials.

Administrator Operations

- System monitoring: When a Vector Store API is used, a session is established to the database and the data insights service. This allows the admin to do the following:
  - Ensure the data insights service is running
  - List active sessions and terminate a session, if needed.
- Managing vector store: Perform the following operations.
  - Create vector stores.
  - Update a vector store that you have admin privileges for.
  - View a list of all vector stores
  - Create and delete a pattern. See Using Patterns in Creating a Metadata based Vector Store for more details.
    Patterns are shared among all admins and any admin can update any pattern.
  - Get the object matching the pattern.
  - List available patterns for creating a metadata-based vector store.
- Manage users
  - Give read and write permissions to the user for all the tables in a vector store.
- List down the users and the corresponding permissions they have on a specific vector store.

User Role in Vector Store

As a user you can access vector stores within your roles and permissions and can do the following:

- Check the health of data insights.
- View the list of vector stores you have access to.
- Do similarity search and ask questions.

Authenticating Users

The administrator authenticates other users to use all the tables in the vector store. The administrator can manage both Read and Write permissions for users.

Every database user has permissions to create vector store on data they have access to. Once the vector store is created, they become the owner/admin and have the read and write access to the vector store they created.

This admin can use the following functions to authenticate other users access to the vector store:

The method's create(), authenticate(), update(), and destroy() need write permissions to the vector store.

The APIs similarity_search(), prepare_response(), and ask() need read permissions to the vector store.

To provide access to a vector store:

1. Instantiate an existing vector store. Assuming that the vector store vs_update exists, use vs = VectorStore("vs_update").
2. Display a list of all users and the respective access rights (read, write) they have on the vector store using:
   Python

   ```
   vs.list_user_permissions()
   ```

   Admins can grant or revoke USER permission for specific users to control their access level to the vector store. The following examples show how the USER permission for user `alice` are handled.

3. Use the grant.user() method to assign USER permission to user alice.
   Python

   ```
   vs.grant.user('alice')
   ```

4. Use the revoke.user() method to remove USER permission from user alice.
   Python

   ```
   vs.revoke.user('alice')
   ```

   To manage the ADMIN access for a user, admins can use the grant.admin() and revoke.admin() methods. This allows them to control who can modify the contents of the vector store.

5. Use the grant.admin() and revoke.admin() methods on user 'oaf' to grant / revoke ADMIN permission.

Python

```
vs.grant.admin('oaf')
vs.list_user_permissions()
```

```
vs.revoke.admin('oaf')
vs.list_user_permissions()
```

Setting up Vector Store

Set up the vector store prior to using it. The following sections provide more details:

- Software Requirements
- Required Privileges
- Importing Modules Required for Vector Store
- Accessing Vector Store APIs Using teradatagenai and REST APIs

Software Requirements

Teradata Vantage Software

You can use vector store on VantageCloud Lake, VantageCloud Enterprise, and VantageCore IntelliFlex. Contact your account team for more details.

For VantageCloud Lake you must have Teradata Vector Store and Data Insights.

Client Software

You need the following software to use vector stores.

Python

Make sure that Python 3.9 or later is installed on your system. You also need these Python packages (available only for VantageCloud Lake):

- teradatagenai - Teradata Package for Generative AI.
- teradataml - Teradata Package for Python

See Accessing Vector Store APIs Using teradatagenai and REST APIs for more details.

Development Environments

Make sure the development environments are compatible with Teradata Package for Generative AI:

- Jupyter
- Spyder

Minimum system requirements

This is the minimum system configuration you must have:

- Windows 7 (64 bit) or later
- macOS 10.9 (64 bit) or later
- Red Hat 7 or later
- Ubuntu 16.04 or later
- CentOS 7 or later
- SLES 12 or later

Required Privileges

To use vector store, you need a DB user with the TD_AIAdmin role and authenticate the user to Cloud Control Panel (CCP) using the URL, PAT access token, and the PEM file. See Create a Personal Access Token for more details.

Importing Modules Required for Vector Store

Before using vector store, you need to import modules necessary for using vector store, establishing connection to the database, and to authenticate to CCP.

Although AI Factory (AI-ON-PREM) is not in purview of this guide, some examples are given here for reference. See Teradata® AI Factory for more details.

1. Import necessary modules from teradataml and teradatagenai.
   - From teradatagenai, import VSManager, VectorStore, and VSPattern.
   - From teradataml, import create_context and set_auth_token
2. Establish a connection to database using create_context.

   ```
   create_context(host='<host>', username='<username>', password='<password>')
   ```

3. Generate token to access services running on Teradata Vantage using set_auth_token.

   For VantageCloud Lake, generate token to authenticate to CCP using the following steps:

a. Get the base_url, pat_token, and pem file. See Required Privileges for more details.
   You can get the base_url from the Open Analytics endpoint URL by removing the term open-analytics.
b. For AI-ON-PREM, you can access the vector store services using Basic authentication as well as keycloak IDP.
   Python - VCL

      Using PAT and PEM

```
set_auth_token(base_url='<base_url>', pat_token='<pat_token>', pem_file='<.pem file>')
```

   REST API - VCL

      Using JWT

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/session" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: application/json' \
   -d '{
         "database_name": "oaf"
       }' \
   -c session_cookie \
   -s
```

   Python - AI-on-Prem

      Using user name and password

```
import getpass
set_auth_token(base_url=getpass.getpass("base_url : "),
               username=getpass.getpass("username : "),
               password=getpass.getpass("password : "))
```

      When IDP is keycloak

```
import getpass
set_auth_token(base_url=base_url, auth_url=auth_url, password=password)
```

      where auth_ur“ is the keycloak URL.

   REST API - AI-on-Prem

      Using user name and password

```
encoded_credentials=$(echo -n "$USERNAME:$PASSWORD" | base64)
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/session" \
   -H "Authorization: Basic $encoded_credentials" \
   -H 'Content-Type: application/json' \
   -d '{
         "database_name": "oaf"
       }' \
   -c session_cookie \
   -s
```

      Using JWT token generated by keycloak

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/session" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: application/json' \
   -d '{
         "database_name": "oaf"
       }' \
   -c session_cookie \
   -s
```

After performing the previous steps, you can check the health of the service:

Python

```
VSManager.health()
```

REST API

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/health" \
   -H "Authorization: Bearer $JWT" \
```

You can also check the list of available vector stores.

Python

```
VSManager.list()
```

REST API

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/vectorstores" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Accessing Vector Store APIs Using teradatagenai and REST APIs

teradatagenai

You can access Vector store APIs using teradatagenai, the Teradata Python Package for Generative AI. For your convenience, the Python files are available as attachment with this guide. For more details on the Generative AI, see Teradata Package for Generative AI.
The teradatagenai package is only supported in VantageCloud Lake.

The related Python Generative AI files are attached with this guide. See Teradata Package for Generative AI and Teradata Package for Generative AI Function Reference on VantageCloud Lake for detailed description and usage examples of the teradataml functions.

REST APIs

You can access Vector store related REST APIs available as attachment in this guide.
Vector store REST APIs are available in VantageCloud Enterprise and AI Factory).

Creating a Vector Store

You can create a vector store in the following ways.

- Creating a Content Based Vector Store
- Creating a Metadata based Vector Store
- Creating a File-based Vector Store
- Creating an Embedding-based Vector Store
- Searching in a Vector Store

Creating a Content Based Vector Store

Content-based vector store uses table contents as vector store and is useful for cases where table contents contain documents, texts of data that need to be searched to find relevant answers.

You can create a vector store either using contents in a database table or from a PDF.

Using contents in a table

In the following examples, a vector store named vs_example1 is created using contents from table amazon_reviews_25.

Python - AWS Bedrock

```
# Python
vs = VectorStore('vs_example1')
vs.create(embeddings_model='amazon.titan-embed-text-v1',
          chat_completion_model='anthropic.claude-instant-v1',
          search_algorithm='HNSW',
          seed=10,
          top_k=10,
          ef_construction=32,
          num_connpernode=32,
          maxnum_connpernode=32,
          metric='EUCLIDEAN',
          apply_heuristics=False,
          ef_search=32,
          target_database='oaf',
          object_names='amazon_reviews_25',
          key_columns=['rev_id', 'aid'],
          data_columns=['rev_text'],
          vector_column='VectorIndex')
```

Create the Vector Store with embeddings_model amazon.titan-embed-text-v1 and chat_completion_model anthropic.claude-instant-v1 using name argument.

```
# Python
vs = VectorStore()
```

```
vs.create(name = 'vs_example1',
          embeddings_model='amazon.titan-embed-text-v1',
          chat_completion_model='anthropic.claude-instant-v1',
          search_algorithm='HNSW',
          seed=10,
          top_k=10,
          ef_construction=32,
          num_connpernode=32,
          maxnum_connpernode=32,
          metric='EUCLIDEAN',
          apply_heuristics=False,
          ef_search=32,
          target_database='oaf',
          object_names='amazon_reviews_25',
          key_columns=['rev_id', 'aid'],
          data_columns=['rev_text'],
          vector_column='VectorIndex')
```

The following output appears:

Vector store vs_example1 creation started.

Use the status() API to check the status of the operation.

.

REST API - AWS Bedrock

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_example1" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "target_database": "oaf",
         "object_names": ["amazon_reviews_25"],
         "key_columns": ["rev_id", "aid"],
         "data_columns": ["rev_text"],
         "vector_column": "VectorIndex"
       }' \
   -F 'vs_parameters={
         "embeddings_model": "amazon.titan-embed-text-v1",
         "chat_completion_model": "anthropic.claude-instant-v1",
         "search_algorithm": "HNSW",
         "seed": 10,
         "top_k": 10,
         "ef_construction": 32,
         "num_connPerNode": 32,
         "maxNum_connPerNode": 32,
         "metric": "EUCLIDEAN",
         "apply_heuristics": false,
         "ef_search":32
       }' \
   -b session_cookie \
   -s
```

Python - Azure Open AI

```
# Python
vs = VectorStore('vs_example1')
vs.create(embeddings_model= 'text-embedding-ada-002',
          chat_completion_model= 'gpt-35-turbo-16k',
          search_algorithm= 'HNSW',
          seed=10,
          top_k=10,
          ef_construction=32,
          num_connpernode=32,
          maxnum_connpernode=32,
          metric='EUCLIDEAN',
          apply_heuristics=False,
          ef_search=32,
          object_names= 'amazon_reviews_25',
          key_columns= ['rev_id', 'aid'],
          data_columns= ['rev_text'],
          vector_column= 'VectorIndex')
```

Python - Nvidia NIM

The NIM models are only available for usage on AI-ON-PREM systems.

Create the vector store with embeddings_model nvidia/nv-embedqa-e5-v5 and chat_completion_model mistralai/mistral-7b-instruct-v0.3.

Once the vector store is created, supply the endpoint details in emdeddings_base_url and completions_base_url.

```
vs = VectorStore('vs_example1')
vs.create(embeddings_model= 'nvidia/nv-embedqa-e5-v5',
          chat_completion_model='mistralai/mistral-7b-instruct-v0.3',
          chat_completion_max_tokens=8192,
          embeddings_dims=1024,
          search_algorithm= 'VECTORDISTANCE',
          top_k=10,
          object_names= 'amazon_reviews_25',
          key_columns= ['rev_id', 'aid'],
          data_columns= ['rev_text'],
          emdeddings_base_url='https://gpudev1-modelops.labs.teradata.com/serving/65bb272e-1200-464c-9e49-c2165e77966f/v1',
          completions_base_url='https://gpudev1-modelops.labs.teradata.com/serving/d5f1aec4-a9af-46b5-aeae-c1320aa506c0/v1'
          )
```

Using Contents from a PDF

In the following example, a vector store named vs_pdf is created by using contents from PDF files.

Here the document_files parameter contains a list of files. In case the file is not in the current directory, you must provide a fully qualified name of the file.

The files are processed internally, converted to chunks, and stored into a database table. Alternatively, you can choose to chunk their files themselves, stored them into a database table and specify the details of that table. When the document_files parameter is specified, you also must provide database_name, object_name, data_columns where the file contents are stored.

The following is an example of creating a content-based Vector store.

Python

```
import os
files = [os.path.join(os.getcwd(), "SQL_Fundamentals.pdf")]
pdf_vs = VectorStore('vs_pdf')
pdf_vs.create(embeddings_model='amazon.titan-embed-text-v1',
              chat_completion_model='anthropic.claude-instant-v1',
              search_algorithm='VECTORDISTANCE',
              top_k=10,
              target_database='oaf',
              object_names='sql_fundamentals',
              data_columns=['chunks'],
              vector_column='VectorIndex',
              chunk_size=500,
              optimized_chunking=False,
              document_files=files)
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_pdf" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "target_database": "oaf",
         "object_names": ["sql_fundamentals"],
         "data_columns": ["chunks"],
         "vector_column": "VectorIndex",
         "chunk_size": 500,
         "optimized_chunking": false
       }' \
   -F 'vs_parameters={
         "embeddings_model": "amazon.titan-embed-text-v1",
         "chat_completion_model": "anthropic.claude-instant-v1",
         "search_algorithm": "VECTORDISTANCE",
         "top_k": 10
       }' \
   -F 'document_files=@SQL_Fundamentals.pdf;type=application/pdf' \
   -b session_cookie \
   -s
```

Creating a Metadata based Vector Store

The metadata-based vector store contains metadata information about tables such as the table and column description. The description is populated using an LLM through a prompt and optionally samples from table contents and other tables in the vector store are used to generate it. You can create the metadata based vector store using tables or patterns.

Using Tables

You can specify the table names in include_objects parameter which you need to include while creating a vector store and if you need to exclude specific set of tables, you can specify the same in exclude_objects parameter.

Python

```
vs_meta = VectorStore('vs_meta')
vs_meta.create(embeddings_model= 'amazon.titan-embed-text-v1',
               search_algorithm= 'VECTORDISTANCE',
               top_k= 10,
               target_database = 'semantic_data',
               include_objects = ['cricket', 'test'],
               exclude_objects = ['football'])
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_meta" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "target_database": "semantic_data",
         "include_objects": ["cricket", "test"],
         "exclude_objects": ["football"]
       }' \
   -F 'vs_parameters={
         "embeddings_model": "amazon.titan-embed-text-v1",
         "search_algorithm": "VECTORDISTANCE",
         "top_k": 10
       }' \
   -b session_cookie \
   -s
```

Using Patterns

Specify the patterns in include_patterns and exclude_patterns parameter.

1. Create two patterns.
   Python

   ```
   # This pattern will fetch all tables/views in 'SEMANTIC_DATA' database which start with name 'CRICKET'.
   ptn1 = VSPattern(name='cricket_include')
   ptn1.create(pattern_string = 'SEMANTIC_DATA.CRICKET%')
   ```

   ```
   # This pattern will fetch all tables/views in 'SEMANTIC_DATA' database which start with name 'Test'.
   ptn2 = VSPattern(name='cricket_exclude')
   ptn2.create(pattern_string='SEMANTIC_DATA.%Test%')
   ```

   REST API

   ```
   # This pattern will fetch all tables/views in 'SEMANTIC_DATA' database which start with name 'CRICKET'.
   curl -X 'POST' \
      "$BASE_URL/data-insights/api/v1/patterns/cricket_include?pattern_string=SEMANTIC_DATA.CRICKET%" \
      -H "Authorization: Bearer $JWT" \
      -b session_cookie \
      -s
   ```

   ```
   # This pattern will fetch all tables/views in 'SEMANTIC_DATA' database which contain 'Test' in their name.
   curl -X 'POST' \
      "$BASE_URL/data-insights/api/v1/patterns/cricket_exclude?pattern_string=SEMANTIC_DATA.%Test%" \
      -H "Authorization: Bearer $JWT" \
      -b session_cookie \
      -s
   ```

2. Create an instance of the VectorStore class with name 'cricket_test_example'. Call the create method with ptn1 passed in include_patterns argument which specifies to use all objects matching this pattern while creating the VectorStore and ptn2 passed in exclude_patterns which specifies all objects matching this pattern should not be used while creating the vector store.

   Python

```
vs_meta = VectorStore(name='cricket_test_example')

vs_meta.create(embeddings_model= 'amazon.titan-embed-text-v1',
               search_algorithm= 'VECTORDISTANCE',
               top_k= 10,
               target_database = 'semantic_data',
               include_patterns = [ptn1],
               exclude_patterns = [ptn2])
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/cricket_test_example" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "target_database": "semantic_data",
         "include_patterns": ["cricket_include"],
         "exclude_patterns": ["cricket_exclude"]
       }' \
   -F 'vs_parameters={
         "embeddings_model": "amazon.titan-embed-text-v1",
         "search_algorithm": "VECTORDISTANCE",
         "top_k": 10
       }' \
   -b session_cookie \
   -s
```

For more details on Patterns, see Managing Vector Store Patterns.

Managing Vector Store Patterns

For metadata-based vector stores, the selection of tables/views can be huge. They can span multiple databases and it can become tedious to list them using include_objects and exclude_objects as mentioned in Ingest support. Patterns provide a way to select these tables/views and columns using simple regular expressions. A pattern string can be formed by using SQL wildcards (% or _) and is specified while creating a pattern.

For example:

- A pattern created using pattern_string - SEMANTIC_DATA.CRICKET% will internally fetch all tables and views in the SEMANTIC_DATA database starting with CRICKET at the time of vector store creation.
- A pattern created using pattern_string - log__ will internally fetch all tables and views starting with log and having two extra characters like log_a or log12 in the logged-in database at the time of vector store creation.
- A pattern created using pattern_string - data_.% will internally fetch all tables and views in all databases starting with data and having one extra character like data1.t1, data2.v2, datax.train at the time of vector store creation.

Patterns are kind of regex which is used for combining names of tables or views patching the pattern string which can then be used for creating metadata based vector store.

You can list all available patterns in the vector store using the following syntax. You'll get a DataFrame containing the name and the pattern string for all available patterns:

Python

```
from teradatagenai import VSManager
VSManager.list_patterns()
```

REST API

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/patterns" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Instantiate VSPattern

This example instantiates a pattern named 'cricket_include'. If the pattern already exists, user can use this object or else use create to create the same.

Python

```
ptn = VSPattern(name='cricket_include')
```

Create a Pattern

Create the pattern by specifying the pattern_string as 'SEMANTIC_DATA.CRICKET%'.

Python

```
ptn.create(pattern_string = 'SEMANTIC_DATA.CRICKET%')
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/patterns/cricket_include?pattern_string=SEMANTIC_DATA.CRICKET%" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Get Matching Objects

Retrieve a list of objects that match the pattern string 'SEMANTIC_DATA.CRICKET%' which will internally fetch all tables in the 'SEMANTIC_DATA' database starting with 'CRICKET' at the time of vector store creation.

Python

```
ptn.get()
```

REST API

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/patterns/cricket_include" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Delete a Pattern

Deletes the pattern

Python

```
ptn.delete()
```

REST API

```
curl -X 'DELETE' \
   "$BASE_URL/data-insights/api/v1/patterns/cricket_include" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Creating a File-based Vector Store

You can create a file-based vector store by passing files or directory paths to the document_files parameter. You can also pass the paths as wildcards.

Currently only PDF files are supported as input.

The files are processed internally, converted to chunks and stored into a database table. In case of file-based vector store, only one object_names should be specified that indicates the file content splits.

key_columns is not supported for file-based vector stores.

You can use additional arguments specific to NIM to customize the parsing of input files.

In the following example, a Vector store named vs_pdf is created by using contents from PDF files.

Here the document_files parameter contains a list of files. In case the file is not in the current directory, you must provide the fully qualified name of the file. Along with the document_files parameter, you must also provide database_name, object_name, data_columns to indicate where the file content splits should be stored.

Python

```
import os
files = [os.path.join(os.getcwd(), "SQL_Fundamentals.pdf")]
pdf_vs = VectorStore('vs_pdf')
pdf_vs.create(embeddings_model='amazon.titan-embed-text-v1',
              chat_completion_model='anthropic.claude-instant-v1',
              search_algorithm='VECTORDISTANCE',
              top_k=10,
              target_database='oaf',
              object_names='sql_fundamentals',
              data_columns=['chunks'],
              vector_column='VectorIndex',
              chunk_size=500,
              optimized_chunking=False,
              document_files=files)
```

Creating a VectorStore vs_pdf using the name parameter in create().

```
import os
files = [os.path.join(os.getcwd(), "SQL_Fundamentals.pdf")]
pdf_vs = VectorStore()
pdf_vs.create(name='vs_pdf',
              embeddings_model='amazon.titan-embed-text-v1',
              chat_completion_model='anthropic.claude-instant-v1',
              search_algorithm='VECTORDISTANCE',
              top_k=10,
              target_database='oaf',
              object_names='sql_fundamentals',
              data_columns=['chunks'],
              vector_column='VectorIndex',
              chunk_size=500,
              optimized_chunking=False,
              document_files=files)
```

The following appears:

Vector store vs_pdfcreation started.

Use the status() API to check the status of the operation.

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_pdf" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "target_database": "oaf",
         "object_names": ["sql_fundamentals"],
         "data_columns": ["chunks"],
         "vector_column": "VectorIndex",
         "chunk_size": 500,
         "optimized_chunking": false
       }' \
   -F 'vs_parameters={
         "embeddings_model": "amazon.titan-embed-text-v1",
         "chat_completion_model": "anthropic.claude-instant-v1",
         "search_algorithm": "VECTORDISTANCE",
         "top_k": 10
       }' \
   -F 'document_files=@SQL_Fundamentals.pdf;type=application/pdf' \
   -b session_cookie \
   -s
```

Creating an Embedding-based Vector Store

You can create an embedding-based vector store by supplying the user-generated embedded data to object_names. It is mandatory to set the is_embedded parameter to True if the input data is pre embedded.

You can get the embeddings in the following ways:

- Bring your own embedded data to vector store.
- TextAnalayticsAI to generate embeddings.

In the following example, the embeddings are generated using TextAnalyticsAI and an embedding-based vector store “vs_TAI_emb” is created from the generated data.

Python AWS

```
import os, getpass
from teradatagenai import TeradataAI, TextAnalyticsAI, load_data
load_data('employee', 'employee_data')
data = DataFrame('employee_data')

# Set the environment variables related to the authorization arguments.
os.environ['AWS_ACCESS_KEY_ID'] = getpass.getpass("Enter AWS Access Key ID: ")
os.environ['AWS_SECRET_ACCESS_KEY'] = getpass.getpass("Enter AWS Secret Access Key: ")
os.environ['AWS_DEFAULT_REGION'] = getpass.getpass("Enter AWS Default Region: ")

llm_embedding = TeradataAI(api_type = "aws",
                  model_name = "amazon.titan-embed-text-v2:0")
# Instantiate the TextAnalyticsAI class with the embedding model.
obj_embeddings = TextAnalyticsAI(llm=llm_embedding)
# Generate embeddings for the articles in the employee data.
TAI_embeddings = obj_embeddings.embeddings(column="articles", data=data, accumulate='articles',output_format='VECTOR')
```

```
vs2 = VectorStore("vs_TAI_emb" , log = True)
vs2.create(object_names= TAI_embeddings,
           data_columns= ['Embedding'],
           vector_column = 'VectorIndex',
           is_embedded = True)
```

In the following example, the pre embedded table present in the database is directly passed as an input to object_names.

Python-NIM

```
vs2 = VectorStore("vs_TAI_emb" , log = True)
vs2.create(object_names= "amazon_reviews_embedded",
           key_columns= ['rev_id', 'aid'],
           data_columns= ['embedding'],
           vector_column = 'VectorIndex',
           is_embedded = True)

vs2.destroy()
```

Searching in a Vector Store

Once the vector store is initialized, you can perform various types of searchers, such as, similarity_search, similarity_search_by_vector, prepare_response, and ask questions on the contents of the vector store. You can search either in interactive or batch modes.

The batch mode is only applicable for AWS. Also, the Metadata based vector store does not currently support prepare_response() and ask() APIs. Only similarity_search is supported.

The key vector store ingredients work at a much higher speed when work in batch mode compared to interactive mode. These ingredients include:

- Embedding generation (AI_TextEmbedding via AWS Titan / Azure embedding models)
- Vector Search (TD_VectorDistance, TD_KMeans, TD_HNSW)
- Response generation via LLM (AWS Anthropic / Azure LLMs)

The embedding generation will always run in batch mode because of its nature whereas for vector search and response generation, you can opt either for interactive or batch mode.

To run in batch mode, you must provide the following parameters:

```
batch_input_table: Specifies the table name or teradataml
             DataFrame to be indexed for batch mode. batch_input_id_column: Specifies the ID column to be indexed for batch mode.
           batch_input_query_column: Specifies the query column to be indexed for batch
           mode.
```

Assuming you are using an already existing vector store vs_update , initialize the vector store:

```
# Initialize existing VS updated_vs =
    VectorStore("vs_update")
```

similarity_search

The similarity search uses the algorithm specified in search_algorithm. It is used to perform the search against the vector store.

By default, the return type is teradataml DataFrame, however you have the option to choose between JSON and pandas DataFrame which results in quicker retrieval.

Interactive mode: The result contains top_k rows (which is an argument in create()) along with similarity score found by the search_algorithm and is most relevant to the question based on the specified metric or vector store also support search threshold criteria. When search_threshold is supplied, the results with similarity score above the search threshold is returned.
The following example finds the For example, the find top_k similar responses to the question asked:

Python

```
question = 'Which book are all the reviews talking about?'
response = updated_vs.similarity_search(question=question)
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_update/similarity-search?question=Which%20book%20are%20all%20the%20reviews%20talking%2
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Batch mode

The result contains top_k rows (which is an argument in create()) along with the similarity score found by the search_algorithm to be most relevant to the questions based on the specified metric for all the input questions that must be provided in a table. When search_threshold is supplied, the results with similarity score above the search threshold are returned for each question.

For example, perform a batch similarity search in the vector store.

Here we provide all the questions (>= 100) and we want to search on the vector store in the form of a table home_depot_train. The product_uid column contains the question id and search_term column contains the question. All these questions are sent to an s3 bucket in the form of a json and then embeddings for all the questions are generated by an embeddings model using bedrock batch inference job and the resulting embeddings are downloaded back from s3. After that the embeddings are ran through the search algorithms in database and output is returned.

Python

Initialize vector store:

```
vs = VectorStore('ms_marco_mrr')
```

Create an input Data Frame:

```
df = DataFrame(in_schema("oaf","home_depot_train"))
```

Trigger similarity_search with batch related params:

```
vs.similarity_search(batch_data=df,
                     batch_id_column="product_uid",
                     batch_query_column="search_term")
```

Check the status:

```
vs.status()
```

Once status is `SIMILARITY SEARCH BATCH COMPLETED`, trigger get_batch_result() to get the results.

```
from teradatagenai import VSApi
vs.get_batch_result(VSApi.SimilaritySearch)
```

To get results in JSON format:

```
# Get the result in JSON format.
from teradatagenai import VSApi
ss = vs.get_batch_result(api_name=VSApi.SimilaritySearch, return_type="json")
ss.similar_objects
```

REST API

Trigger similarity search batch:

```
curl -X 'POST' \ "$BASE_URL/data-insights/api/v1/vectorstores/ms_marco_mrr/similarity-search-batch" \ -H "Authorization: Bearer $JWT" \ -
```

Check status:

```
curl -X 'GET' \ "$BASE_URL/data-insights/api/v1/vectorstores/ms_marco_mrr" \ -H "Authorization: Bearer $JWT" \ -b session_cookie \ -s
```

Get results:

```
curl -X 'GET' \ "$BASE_URL/data-insights/api/v1/vectorstores/ms_marco_mrr/similarity-search-batch" \ -H "Authorization: Bearer $JWT" \ -b
```

similarity_search_by_vector

This method performs similarity search with an input pre embedded user query or the table containing the pre embedded query.

The algorithm specified in search_algorithm is used to perform search against the vector store. The result contains the 'top_k' rows along with similarity score

By default, the return type is teradataml DataFrame, however you can choose between json and pandas DataFrame which results in faster retrieval

You can pass the embedded query either directly as the question argument or as a single-row table containing the embedded question. In this case, the name of the table is passed to data and the name of the column containing the embedded question is passed to column.

The following example shows the embedded query directly passed to the question argument.

Python

```
response = vs.similarity_search_by_vector(question='-0.06944,0.080352,0.045963,0.006985,-0.000496,-0.025461,0.045302,-0.028107,0.031
```

In this example, the table containing the embedded vector is passed to data and the column name containing the embeddings is passed to column.

Python

```
response = vs.similarity_search_by_vector(data ='amazon_review_query',column = 'queryEmbedding')

response.similar_objects
```

prepare_response

The prepare_respose takes the result of similarity_search and returns a natural language response which is easier to understand while answering the question given to it. The response is generated by a language model which is configured using chat_completion_model specified while creating the vector store using a pre-configured prompt.

An optional parameter prompt can be used to specify a customized prompt that replaces the internal prompt.

For example, using the similarity_search result generated previously to answer the question in a conversational manner as specified in the prompt.

Python

```
question='Did any one feel the book is thin?'
prompt='Format the response in a conversational way.'
updated_vs.prepare_response(question=question, similarity_results=response, prompt=prompt)
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_update/prepare-response" \
   -H "Authorization: Bearer $JWT" \
   -H "Content-Type: application/json" \
   -d '{
         "question": "Did any one feel the book is thin?",
         "similar_objects": [
            {
               "score": 0.3779027290161969,
               "DataBaseName": "oaf",
               "TableName": "amazon_reviews_25",
               "rev_id": "AENNW2G826191",
               "aid": "000100039X",
               "rev_text": "Its a thin book, very readable and has interesting 1-2 page thoughts on various entities like anger, children
            },
            {
               "score": 0.14626516502219164,
               "DataBaseName": "oaf",
               "TableName": "amazon_reviews_25",
               "rev_id": "A26GKZPS079GFF",
               "aid": "000100039X",
               "rev_text": "I would have to say that this is the best book I have ever read.. I could feel every word deep in my heart ev
            },
            {
               "score": 0.135972999110996,
               "DataBaseName": "oaf",
               "TableName": "amazon_reviews_25",
               "rev_id": "A3V1MKC2BVWY48",
               "aid": "000100039X",
               "rev_text": "Reading this made my mind feel like a still pool of water, cool and quiet in a mossy grotto. It is direct and
            }
         ],
         "prompt": "Format the response in a conversational way."
      }' \
   -b session_cookie \
   -s
```

As another example, perform batch prepare response. This takes in the similarity search object from similarity_search results.

Here you provide all the questions in the form of a table home_depot_train along with the similarity search results obtained from the similarity search job. These results are sent to s3 along with the questions in the form of a json and then responses for all the questions are generated by a chat LLM using bedrock batch inference job. The resulting model output is downloaded back from s3 and responses are returned to the user.

Python

Trigger prepare_response in batch mode. It uses the output from similarity_search

```
prompt= """Structure response in question-answering format
                      Question:             Answer:         """ vs.prepare_response(batch_data="home_depot_train",                    b
```

Check the status:

```
vs.status()
```

Once the status is PREPARE RESPONSE BATCH PROCESSING COMPLETED, trigger get_batch_result to get the results.

ask

ask combines the operation of similarity_search and prepare_response into one call for faster response time.

For example, you can ask the model to format the output as per the custom_prompt supplied while answering the given question.

Python

```
custom_prompt = """List good reviews about the books. Do not assume information. Only provide information that is present in the dat
Format results like this:
Review ID:
```

```
Author ID:
Review:
"""
question ='Are there any reviews saying that the books are inspiring?'
updated_vs.ask(question=question, prompt=custom_prompt)
```

REST API

```
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_update/ask" \
   -H "Authorization: Bearer $JWT" \
   -H "Content-Type: application/json" \
   -d '{
         "question": "Are there any reviews saying that the books are inspiring?",
         "prompt": "List good reviews about the books. Do not assume information. Only provide information that is present in the da
       }' \
   -b session_cookie \
   -s
```

Perform ask in batch mode

Here we provide all the questions (>= 100) we want to search on the vector store in the form of a table home_depot_train. The product_uid column contains the question id and search_term column contains the question. All these questions are sent to an s3 bucket in the form of a json and then embeddings for all the questions are generated by an embeddings model using bedrock batch inference job and the resulting embeddings are downloaded back from s3. After that the embeddings are ran through the search algorithms in database and the similarity search results are obtained. These results are then sent back to s3 along with the questions in the form of a json and then responses for all the questions are generated by a chat llm using bedrock batch inference job. The resulting model output is downloaded back from s3 and responses are returned to the user.

Python

Trigger ask in batch mode.

```
prompt = """Structure the response briefly in 1-2 lines."""
vs.ask(batch_data="home_depot_train",
       batch_id_column="product_uid",
       batch_query_column="search_term",
       prompt=prompt)
```

Check the status.

```
vs.status()
```

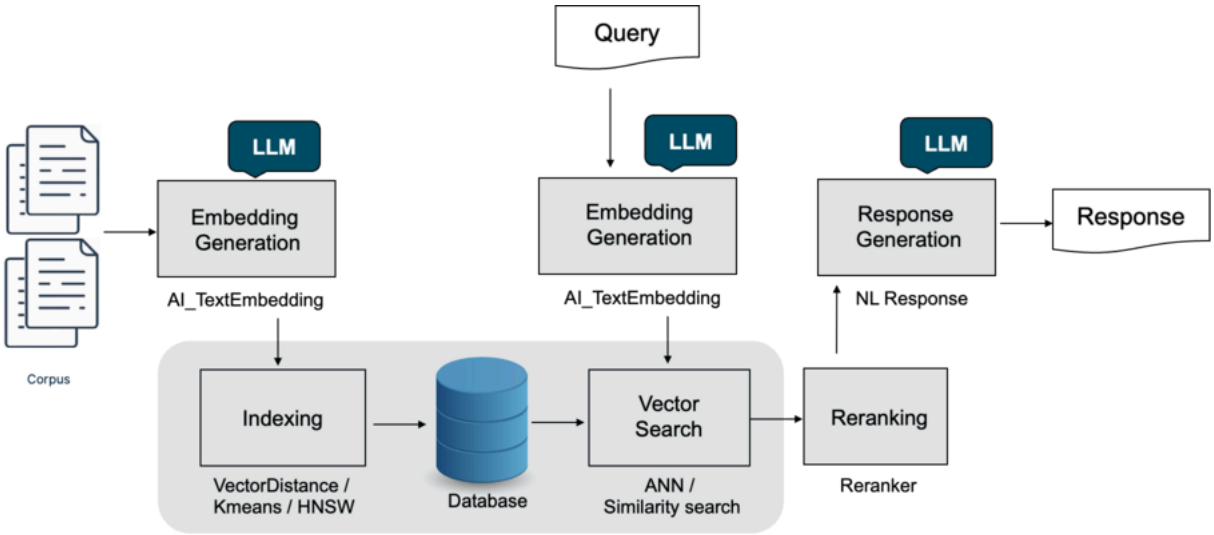
Retrieve the results.

```
batch_ask = vs.get_batch_result(api_name=VSApi.Ask)
```

Using Retrieval-Augmented Generation (RAG)

One of the primary use of vector store is Retrieval-Augmented Generation (RAG), or the ability to ask a text question on the data and have the answer returned in natural language.

This following figure shows the major components of RAG built around vector store.



LLMs like ChatGPT do not have real-time updates and often provide incorrect information when answering questions. A vector database helps by:

- Providing up-to-date, contextually relevant information

- Reducing incorrect information by verifying responses with factual data
- Enhancing responses with domain-specific knowledge

For example:

- A legal chatbot uses vector search to retrieve relevant case laws before answering a legal query.
- A medical AI assistant fetches accurate health articles to support patient queries.

The two steps of RAG are:

1. Upload a PDF. You can upload either using the PDF upload API or fastload and chunked within the database.
2. Search

Detecting Anomalies

Vector stores play a crucial role in anomaly detection by enabling efficient similarity search and pattern recognition across high-dimensional data. By leveraging vector embeddings, anomaly detection systems can identify outliers, fraud, and unusual behaviors in real-time.

Traditional anomaly detection methods rely on rule-based approaches or statistical models, which may struggle with:

- High-dimensional data
- Dynamic patterns that change over time
- Complex, unstructured data like images, text, or logs

Vector stores offer scalable and ML-driven anomaly detection by:

- Identifying outliers based on distance metrics
- Handling high-dimensional, unstructured data, such as images, logs, and transactions
- Performing real-time, large-scale anomaly detection

Creating a Vector Store Using From Methods

You can now use teradatagenai to create vector store in a flexible way from different data sources. Based on your data format and needs, you can choose from the following creation methods:

- File-based Vector Store (from_documents): Create a vector store from PDF documents.
- Content-Based Vector Store from Raw Text (from_texts): Create a vector store from text strings.
- Content-Based Vector Store from Tables (from_datasets): Create a vector store from tables or DataFrames.
- Embedding-Based Vector Store (from_embeddings): Create a vector store from pre-computed embeddings.

Each method targets specific use cases, providing solutions for various data formats and requirements. Common parameters are shared across all methods, while specific arguments are unique to individual methods.

Common Parameter Tables

The following are the common parameter applicable for various types of vector store creating using from_methods.

Generic Parameters

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
description	Optional Argument	Specifies the description of the vector store.	None	None	str	None
target_db	Optional Argument	Specifies the database name where the vector store is created.	None	None	str	If not specified, vector store is created in the database which is in use.
vector_column	Optional Argument	Specifies the name of the column to be used for storing the embeddings.	vector_index	None	str	None

Model Parameters

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
embeddings	Required for NVIDIA NIM, Optional otherwise	Specifies the embeddings model to be used for generating the embeddings.	For AWS: amazon.titan-embed-text-v2:0 For Azure: text-embedding-3-small	For AWS: • amazon.titan-embed-text-v1 • amazon.titan-embed-image-v1 • amazon.titan-embed-text-v2:0 For Azure:	str, TeradataAI	None

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
				• text-embedding-ada-002 • text-embedding-3-small • text-embedding-3-large		
embeddings_dims	Required for NVIDIA NIM, Optional otherwise	Specifies the number of dimensions to be used for generating the embeddings. The value depends on the "embeddings".	For AWS: • amazon.titan-embed-text-v1: 1536 • amazon.titan-embed-image-v1: 1024 • amazon.titan-embed-text-v2: 1024 For Azure: • text-embedding-ada-002: 1536 • text-embedding-3-small: 1536 • text-embedding-3-large: 307	For AWS: • amazon.titan-embed-text-v1: 1536 • amazon.titan-embed-image-v1: [256, 384, 1024] • amazon.titan-embed-text-v2: [256, 512, 1024] For Azure: • text-embedding-ada-002: 1536 only • text-embedding-3-small: 1 <= dims <= 1536 • text-embedding-3-large: 1 <= dims <= 3072	str	None
chat_completion_model	Required for NVIDIA NIM, Optional otherwise	Specifies the name of the chat completion model to be used for generating text responses.	For AWS: anthropic.claude-3-haiku-20240307-v1:0 For Azure: gpt-35-turbo-16k	For AWS: • anthropic.claude-3-haiku-20240307-v1:0 • anthropic.claude-instant-v1 • anthropic.claude-3-5-sonnet-20240620-v1:0 For Azure: • gpt-35-turbo-16k	str, TeradataAI	None
embeddings_bbase_url	Required for NVIDIA NIM, Optional otherwise	Specifies the base URL for the service to be used for generating embeddings.	None	None	str	Applicable only while using NVIDIA NIM endpoints.
completions_base_url	Optional Argument	Specifies the base URL for the service to be used for generating completions.	None	None	str	Applicable only while using NVIDIA NIM endpoints.
chat_completion_max_tokens	Required for NVIDIA NIM, Optional otherwise	Specifies the maximum number of tokens to be generated by the "chat_completion_model".	16384	[1, 16384]	int	None

Creating a File-Based Vector Store using from_documents

Create a new file-based vector store from the PDF documents, directories, or wildcard patterns using the following is the syntax:

```
VectorStore.from_documents(name, documents, embedding=None, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the from_documents.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
name	Required Argument	Specifies the name of the vector store to be created from input document(s).	None	None	str	None

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
documents	Required Argument	Specifies the input dataset in document files format. It can be used to specify input documents in file format. A directory path or wildcard pattern can also be specified. The files are processed internally, converted to chunks and stored into a database table.	None	None	str, list	Only PDF format is currently supported. You can supply multiple document files. Specify the fully qualified file name. Examples include: • 'file1.pdf', 'file2.pdf • /path/to/pdfs • /path/to/pdfs/_.pdf • /path/to/pdfs/**/_.pdf
object_names	Optional Argument	Specifies the table name that is to be used for file content splits.	None	None	str	Only one table name should be specified.
target_database	Optional Argument	Specifies the database name where the file content splits are stored.	None	None	str	If not specified, vector store is created in the database which is in use.
data_columns	Optional Argument	Specifies the column name(s) where the content splits are to be stored.	None	None	str	Only one name should be specified.
chunk_size	Optional Argument	Specifies the number of characters in each chunk to be used while splitting the input file.	512	None	int	Applicable only for 'file-based' vector stores.
optimized_chunking	Optional Argument	Specifies whether an optimized splitting mechanism supplied by Teradata should be used. The documents are parsed internally in an intelligent fashion based on file structure and chunks are dynamically created based on section layout.	False	None	bool	The "chunk_size" field is not applicable when "optimized_chunking" is set to True. Applicable only for 'file-based' vector stores.
nv_ingestor	Optional Argument	Specifies whether to use NVIDIA NV-Ingest for processing the document files.	False	None	bool	Applicable only while using NVIDIA NIM endpoints.Applicable only for 'file-based' vector stores.
display_metadata	Optional Argument	Specifies whether to display metadata describing objects extracted from document files when using NVIDIA NV-Ingest.	False	None	bool	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_text	Optional Argument	Specifies whether to extract text from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_images	Optional Argument	Specifies whether to extract images from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_tables	Optional Argument	Specifies whether to extract tables from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_infographics	Required for NVIDIA NIM, Optional otherwise	Specifies whether to extract infographics from document files.	False	None	bool	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_method	Required for NVIDIA NIM, Optional otherwise	Specifies the method to be used for extracting text from the document files.	pdfium	pdfium, nemoretriever_parse	str	Applicable only for 'file-based' vector stores.Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
tokenizer	Optional Argument	Specifies the tokenizer to be used for splitting the text into chunks.	meta-llama/Llama-3.2-1B	None	str	Applicable only when "nv_ingestor" is set to True and "document_files" are supplied. This applies exclusively when using NVIDIA NIM endpoints.
header_height	Optional Argument	Specifies the height (in points) of the header section of a PDF document to be trimmed before processing the main content. This is useful for removing unwanted header information from	0	None	int	Applicable only for 'file-based' vector stores.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
		each page of the PDF. Recommended value is 55.				
footer_height	Optional Argument	Specifies the height (in points) of the footer section of a PDF document to be trimmed before processing the main content. This is useful for removing unwanted footer information from each page of the PDF. Recommended value is 55.	0	None	int	Applicable only for 'file-based' vector stores.
ingest_host	Required for NVIDIA NIM, Optional otherwise	Specifies the HTTP host for the service to be used for document parsing.	None	None	str	Applicable only while using NVIDIA NIM endpoints.
ingest_port	Optional Argument	Specifies the HTTP port for the service to be used for document parsing.	7670	None	int	Applicable only while using NVIDIA NIM endpoints.

The following is an example:

```
import teradatagenai
from teradatagenai.vector_store import VectorStore
from teradatagenai import TeradataAI
import os
# Example 1: Create from single PDF file
base_dir = os.path.dirname(teradatagenai.__file__)
file_1 = os.path.join(base_dir, 'example-data', 'SQL_Fundamentals.pdf')
vs_instance = VectorStore.from_documents(
    name="vs_example_1",
    documents=file_1,
    embedding="amazon.titan-embed-text-v1"
)
# Example 2: Create from multiple PDF files with TeradataAI embedding
os.environ["AWS_DEFAULT_REGION"] = "<Enter AWS Region>"
os.environ["AWS_ACCESS_KEY_ID"] = "<Enter AWS Access Key ID>"
os.environ["AWS_SECRET_ACCESS_KEY"] = "<Enter AWS Secret Key>"
os.environ["AWS_SESSION_TOKEN"] = "<Enter AWS Session key>"
llm_aws = TeradataAI(api_type="aws", model_name="amazon.titan-embed-text-v2:0")
file_2 = os.path.join(base_dir, 'example-data', 'LLM_handbook.pdf')
files = [file_1, file_2]
vs_instance = VectorStore.from_documents(
    name="vs_example_2",
    documents=files,
    embedding=llm_aws
)
# Example 3: Create from directory with chat completion model
obj = TeradataAI(
    api_type="aws",
    access_key="<AWS bedrock access key>",
    secret_key="<AWS bedrock secret key>",
    session_key="<AWS bedrock session key>",
    region="us-west-2",
    model_name="anthropic.claude-instant-v1",
    model_args={"max_tokens_to_sample": 2048}
)
vs_instance = VectorStore.from_documents(
    name="vs_example_3",
    documents="<Enter/path/to/directory/containing/pdf/files>",
    embedding="amazon.titan-embed-text-v1",
    chat_completion_model=obj
)
```

Creating a Content-Based Vector Store using from_texts

Create a new content-based vector store from raw text or list of texts using the following syntax:

```
VectorStore.from_texts(name, data, embedding=None, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the from_texts.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
name	Required Argument	Specifies the name of the vector store to be created from input list of raw text strings.	None	None	str	None
texts	Required Argument	Specifies the text(s) to be indexed for vector store.	None	None	str, list	None

The following is an example.

```
# Example 1: Create from list of raw strings
vs_instance = VectorStore.from_texts(
    name="vs_example_1",
    texts=["This is a sample text.", "This is another sample text."],
    embedding="amazon.titan-embed-text-v1"
)
```

Creating a Content-Based Vector Store using from_datasets

Create a new content-based vector store from tables or DataFrames using the following syntax:

```
VectorStore.from_datasets(name, data, embedding=None, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the from_datasets.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
name	Required Argument	Specifies the name of the vector store to be created from the input dataset(s).	None	None	str	None
data	Required Argument	Specifies the table name(s)/teradataml DataFrame(s) to be indexed for vector store. Teradata recommends to use teradataml DataFrame as input.	None	None	str, list, DataFrame	Multiple tables/views/DataFrames can be passed in "dataset". If the table is in another database than the database in use, make sure to pass in the fully qualified name or a DataFrame object. If multiple tables/views are passed, each table should have the columns which are mentioned in "data_columns" and "key_columns". When "target_database" is not set and only table name is passed to "dataset", then the input is searched in default database.
data_columns	Required Argument	Specifies the name(s) of the data column(s) to be used for embedding generation(vectorization).	None	None	str, list	When multiple data columns are specified, data is unpivoted to get a new key column "AttributeName" and a single data column "AttributeValue".
key_columns	Optional Argument	Specifies the name(s) of the key column(s) to be used for indexing.	None	None	str, list	None

The following is an example.

```
# Example 1: Create from table name with string embedding
from teradatagenai import VectorStore, load_data
from teradataml import DataFrame
# Load the amazon reviews data
load_data('byom', 'amazon_reviews_25')
vs_instance1 = VectorStore.from_datasets(
    name="vs_example_1",
    data="amazon_reviews_25",
    data_columns=["rev_text"],
    embedding="amazon.titan-embed-text-v1"
)
# Example 2: Create from DataFrame with TeradataAI embedding
from teradatagenai import load_data, TeradataAI
from teradataml import DataFrame
import os
# Initialize the TeradataAI object using environment variables
os.environ["AWS_DEFAULT_REGION"] = "<Enter AWS Region>"
os.environ["AWS_ACCESS_KEY_ID"] = "<Enter AWS Access Key ID>"
os.environ["AWS_SECRET_ACCESS_KEY"] = "<Enter AWS Secret Key>"
os.environ["AWS_SESSION_TOKEN"] = "<Enter AWS Session key>"
llm_aws = TeradataAI(api_type="aws",
                     model_name="amazon.titan-embed-text-v2:0")
load_data('employee', 'employee_data')
data = DataFrame('employee_data')
vs_instance2 = VectorStore.from_datasets(
    name="vs_example_2",
    data=data,
    data_columns=["articles"],
```

```
    embedding=llm_aws
)
```

Creating an Embedding-Based Vector Store using from_embeddings

Create a new embedding-based vector store from pre-embedded tables or DataFrames using the following syntax:

```
VectorStore.from_embeddings(name, data, **kwargs)
```

Parameters

In addition to the Generic Parameters in the Common Parameter Tables, the following are also used in the from_embeddings.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
name	Required Argument	Specifies the name of the vector store to be created from input data.	None	None	str	None
data	Required Argument	Specifies the table name(s)/teradataml DataFrame(s) that are pre embedded to be indexed for vector store. Teradata recommends to use teradataml DataFrame as input.	None	None	str, list, DataFrame	If multiple tables or views are provided, each table must include the columns specified in "data_columns" and "key_columns." When "target_database" is not set and only the table name is passed to "data," the input will be searched in the default database.
data_columns	Required Argument	Specifies the name of the column that contains the pre embedded data.	None	None	str, list	When multiple data columns are specified, data is unpivoted to get a new key column "AttributeName" and a single data column "AttributeValue".
key_columns	Optional Argument	Specifies the name(s) of the key column(s) to be used for indexing.	None	None	str, list	None
is_normalized	Optional Argument	Specifies whether the input contains normalized embeddings.	False	None	bool	None

The following are the examples:

```
# Example 1: Create from pre-embedded table
from teradatagenai import VectorStore, load_data
from teradataml import DataFrame
load_data('amazon', 'amazon_reviews_embedded')
vs_instance = VectorStore.from_embeddings(
    name="vs_example_1",
    data='amazon_reviews_embedded',
    data_columns=['embedding']
)
# Example 2: Create from embeddings generated using TextAnalyticsAI
import os
from teradatagenai import TeradataAI, TextAnalyticsAI, load_data
from teradataml import DataFrame
# Load the employee data
load_data('employee', 'employee_data')
data = DataFrame('employee_data')
# Initialize the TeradataAI object using environment variables
os.environ["AWS_DEFAULT_REGION"] = "<Enter AWS Region>"
os.environ["AWS_ACCESS_KEY_ID"] = "<Enter AWS Access Key ID>"
os.environ["AWS_SECRET_ACCESS_KEY"] = "<Enter AWS Secret Key>"
llm_embedding = TeradataAI(api_type="aws",
                           model_name="amazon.titan-embed-text-v2:0")
# Create an instance of the TextAnalyticsAI class
obj_embeddings = TextAnalyticsAI(llm=llm_embedding)
# Get the embeddings for the 'articles' column in the data
TAI_embeddings = obj_embeddings.embeddings(
    column="articles",
    data=data,
    accumulate='articles',
    output_format='VECTOR'
)
# Create an instance of the VectorStore class
vs_instance = VectorStore.from_embeddings(
    name="vs_example_2",
```

```
    data=TAI_embeddings,
    data_columns=['Embedding']
)
```

Managing a Vector Store

The following sections provide information on various operations that you can perform on vector store to manage it.

- Obtaining Vector Store Details
- Updating a Vector Store
- Deleting a Vector Store
- Managing Sessions of a Vector Store

Obtaining Vector Store Details

Once a vector store is created, you can perform various operations to obtain the relevant details for a vector store. Once you have the necessary details, you can plan for operations, such as, update, delete, search, and so on.

Obtaining the Vector Store type

You can check the type of the vector store created using the store_type attribute of the VectorStore class.

The supported types are: content-based , metadata-based , file-based , embedding-based. The following is the syntax for it.

The following is an example of checking a vector store type.

Python

```
# Initialize the vector store
vs= VectorStore(name='vs_test_embed', log = True)
# Create the vector store(Not required if vector store already present)
vs.create(object_names= DataFrame('amazon_reviews_embedded'),
           data_columns= ['embedding'],
           key_columns = ['rev_id'],
           vector_column = 'VectorIndex',
           is_embedded = True)

# Get the store type (here it should be embedding-based)
vs.store_type
```

Trimming the Headers and Footers of a PDF

You can set the header_height and footer_height values to 0 to skip the trimming of PDFs.

The syntax is:

Python

```
vs_index = {
    'header_height': 50,
    'footer_height': 50
}
```

Obtaining Embedding and Indexing Information

The get_indexes_embeddings() method returns the DataFrame containing embedding and indexing information of the Vector Store

Python

```
vs1 = VectorStore(name='vs_test_model', log = True)
vs1.create(embeddings_model= 'amazon.titan-embed-text-v1',
          search_algorithm= 'HNSW',
          seed=10,
          top_k=10,
          apply_heuristics=False,
          ef_search=32,
          object_names= 'amazon_reviews_25',
          key_columns= ['rev_id', 'aid'],
          data_columns= ['rev_text'],
          vector_column= 'VectorIndex')

vs1.get_indexes_embeddings()
```

Obtaining Model Information

The get_model_info() method returns model specific DataFrame or dict containing DataFrame depending on the search_algorithm.

If the search_algorithm is “KMEANS”, dict returns two tables:

- kmeans_model - Contains the `kmeans_model` information
- centroids_model - Contains the `centroids` information.

If the search_algorithm is “HNSW”, a DataFrame is returned containing:

- hnsw_model- Contains the `hnsw_model` information.

The following example shows the output of get_model_info on a vector store created with search_algorithm as KMEANS.

Python

```
response = vs.get_model_info()

print(response["kmeans_model"])
print(response["centroids_table"])
```

Obtaining Details of a Vector Store

The get.details() method returns a teradataml DataFrame containing the details of the vector store. The details includes, embeddings model, search algorithm, and any other details which the user has setup while creating or updating the vector store.

The following example initializes an already present content-based vector store and calls the get_details() method on it. It then updates the top_k and metric parameters and calls the method again to check if the changes are reflected.

Python

```
# Initialize (or create) a Vector Store
vs = VectorStore(name = "vs_get_details_test")
# Call the get_details() method
vs.get_details()

# Update "mteric" and "top_k" param.
vs.update(metric = "EUCLIDEAN",
          top_k = 5)
vs.get_details()
```

Updating a Vector Store

You can modify the data or parameters in a vector store as needed.

Only manual updates (triggered by user) are currently supported.

Data Update

- For content-based vector store, you can supply a new table with the data to add or remove from the vector store.
- For file-based vector store, you can supply the files to add or remove from the vector store.
- For embedding-based vector store, you can supply a new table containing pre embedded data to add to the vector store or supply an existing table to remove from the vector store.
- For metadata-based vector store, you can add or remove objects (tables/views) from the vector store. You can specify the addition or removal of data using alter_operation

For KMEANS and HNSW, two types of updates are supported using update_style:

- Minor update: In minor update, the existing model is used to establish the association of new data points. This update is relatively quick because you don't have to reconstruct the underlying model (clustering or graph). If the updated data is small, minor update is preferable.
- Major update: In major update, the existing model is updated or reconstructed by retraining the index. This update can be time consuming and is preferable when updated data is significant compared to the original data.

Data update is accompanied with vectorization of the new data using the embedding model. The following example shows the creation of a vector store and using update() to add new tables to the vector store.

The following are some examples:

Python

```
# Create an instance of the VectorStore class.
vs = VectorStore(name="vec1")
# Create the Vector Store.
# Note this step is not needed if vector store already exists.
vs.create(object_names="amazon_reviews_25",
          description="vector store testing",
          key_columns=['rev_id', 'aid'],
          data_columns=['rev_text'],
          vector_column='VectorIndex',
          embeddings_model="amazon.titan-embed-text-v1")
```

```
# Add the object_names of the Vector Store using
# alter_operation and update_style.
vs.update(object_names='amazon_reviews_10_alter',
          alter_operation="ADD",
          update_style="MINOR")
# Delete the object_names of the content-based Vector Store using
# alter_operation and update_style.
vs.update(object_names='amazon_reviews_25',
          alter_operation="DELETE",
          update_style="MAJOR")
```

REST API

```
# Create the Vector Store.
# Note this step is not needed if vector store already exists.
curl -X 'POST' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vec1" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "object_names": ["amazon_reviews_25"],
         "key_columns": ["rev_id", "aid"],
         "data_columns": ["rev_text"],
         "vector_column": "VectorIndex"
       }' \
   -F 'vs_parameters={
         "description": "vector store testing",
         "embeddings_model": "amazon.titan-embed-text-v1"
       }' \
   -b session_cookie \
   -s
# Add the object_names of the Vector Store using
# alter_operation and update_style.
curl -X 'PATCH' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vec1" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "object_names": ["amazon_reviews_10_alter"],
         "alter_operation": "ADD",
         "update_style": "MINOR"
       }' \
   -b session_cookie \
   -s
# Delete the object_names of the content-based Vector Store using
# alter_operation and update_style.
curl -X 'PATCH' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vec1" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_index={
         "object_names": ["amazon_reviews_25"],
         "alter_operation": "DELETE",
         "update_style": "MAJOR"
       }' \
   -b session_cookie \
   -s
```

The following example shows the update for file-based vector store.

Python

```
     # Create an instance of a file based vector stores
     # from a list of pdf files.
     # Assuming the files are present in the path.
     vs = VectorStore(name = "test_file_based")
     files = ["path/to/files/file1.pdf", "path/to/files/file2.pdf"
     # Create the file based store by passing files in "document_files".
     vs.create(document_files = files,
               top_k = 5,
               description = "Test store for file-based updates",
               search_algorithm = "HNSW",)
     vs.status()
```

```
# Update the vectore store by adding another pdf document(passed as a string).
vs.update(document_files = "path/to/files/file3.pdf",
          alter_operation = "ADD")
```

Parameter Update

You can update any of the parameters of a vector store. The change of an embedding model (embeddings_model) and / or embedding dimension (embeddings_dims) is accompanied with new embedding generation of the data. The remaining parameters update does not require vectorization of data. A complete list of parameters available for updates are specified in the API details.

The update() is asynchronous and hence you must use status() to check the status of the update.
For example, if you plan to update search_algorithm parameter for the vector store vs_update, use:

Python

```
updated_vs.update(search_algorithm="KMEANS")
```

To check the status:

```
updated_vs.status()
```

REST API

```
curl -X 'PATCH' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_update" \
   -H "Authorization: Bearer $JWT" \
   -H 'Content-Type: multipart/form-data' \
   -F 'vs_parameters={
         "search_algorithm": "KMEANS"
       }' \
   -b session_cookie \
   -s
```

To check the status:

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_update" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Deleting a Vector Store

You can permanently delete a vector store by using the destroy operation. It deletes the vector store along with its associated data.

Deleting a vector store does not delete the underlying database connection. If required, you can use it to create a new vector store instance.

Use:

Python

```
vs.destroy()
```

To check status:

```
vs.status()
```

REST API

```
curl -X 'DELETE' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_example1" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

To check status:

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/vectorstores/vs_example1" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Managing Sessions of a Vector Store

Vector store sessions are the sessions established to communicate with the vector store service and database.

As an administrator, you can perform these session operations:

List Sessions

Python

```
VSManager.list_sessions()
```

REST API

```
curl -X 'GET' \
   "$BASE_URL/data-insights/api/v1/sessions" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

Disconnect all Sessions

Python

```
VSManager.disconnect()
```

To disconnect a specific session id:

```
VSManager.disconnect(session_id="")
```

REST API

Disconnect an active session:

```
curl -X 'DELETE' \
   "$BASE_URL/data-insights/api/v1/session" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
```

Disconnect a specific session:

```
curl -X 'DELETE' \
   "$BASE_URL/data-insights/api/v1/sessions/489c1d3f-efb8-46b2-b752-ead9c40dfd09" \
   -H "Authorization: Bearer $JWT" \
   -b session_cookie \
   -s
```

When the session_id argument is passed, only that session is disconnected, else all session IDs created during the current Python session are disconnected. Also, only administrators can disconnect sessions created by other users.

Managing Your Vector Store Using Add and Delete Methods

Teradatagenai provides a comprehensive set of methods for managing your Vector Store content, allowing you to keep your data current without recreating the entire store. Each method is tailored to a specific data type and vector store type.

Adding PDFs Using add_documents Method

The add_documents method adds PDF documents to an existing file-based vector store or creates a new vector store if it doesn't exist. The syntax is

```
add_documents(documents, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the add_documents.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
documents	Required Argument	Specifies the files to be updated to the Vector Store. It can be used to specify input documents in file format. A directory path or wildcard pattern can also be specified.	None	None	str, list	None
name	Optional Argument	Specifies the name of the vector store to be created from input document(s).	None	None	str	None
object_names	Optional Argument	Specifies the table name that is to be used for file content splits.	None	None	str	Only one table name should be specified.
target_database	Optional Argument	Specifies the database name where the file content splits are stored.	None	None	str	If not specified, vector store is created in the database which is in use.
data_columns	Optional Argument	Specifies the column name(s) where the content splits are to be stored.	None	None	str	Only one name should be specified.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None
chunk_size	Optional Argument	Specifies the number of characters in each chunk to be used while splitting the input file.	512	None	int	Applicable only for 'file-based' vector stores.
optimized_chunking	Optional Argument	Specifies whether an optimized splitting mechanism supplied by Teradata should be used. The documents are parsed internally in an intelligent fashion based on file structure and chunks are dynamically created based on section layout.	False	None	bool	The "chunk_size" field is not applicable when "optimized_chunking" is set to True. Applicable only for 'file-based' vector stores.
nv_ingestor	Optional Argument	Specifies whether to use NVIDIA NV-Ingest for processing the document files.	False	None	bool	Applicable only while using NVIDIA NIM endpoints. Applicable only for 'file-based' vector stores.
display_metadata	Optional Argument	Specifies whether to display metadata describing objects extracted from document files when using NVIDIA NV-Ingest.	False	None	bool	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_text	Optional Argument	Specifies whether to extract text from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_images	Optional Argument	Specifies whether to extract images from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_tables	Optional Argument	Specifies whether to extract tables from the document files when using NVIDIA NV-Ingest.	True	None	bool	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_infographics	Required for NVIDIA NIM, Optional otherwise	Specifies whether to extract infographics from document files.	False	None	bool	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
extract_method	Required for NVIDIA NIM, Optional otherwise	Specifies the method to be used for extracting text from the document files.	pdfium	pdfium, nemoretriever_parse	str	Applicable only for 'file-based' vector stores. Applicable only while using NVIDIA NIM endpoints and when "nv_ingestor" is set to True.
tokenizer	Optional Argument	Specifies the tokenizer to be used for splitting the text into chunks.	meta-llama/Llama-3.2-1B	None	str	Applicable only when "nv_ingestor" is set to True and "document_files" is supplied. Applicable only while using NVIDIA NIM endpoints.
header_height	Optional Argument	Specifies the height (in points) of the header section of a PDF document to be trimmed before processing the main content. This is useful for removing unwanted header information from each page of the PDF. Recommended value is 55.	0	None	int	Applicable only for 'file-based' vector stores.
footer_height	Optional Argument	Specifies the height (in points) of the footer section of a PDF document to be trimmed before processing the main content. This is useful for removing unwanted footer information from each page of the PDF. Recommended value is 55.	0	None	int	Applicable only for 'file-based' vector stores.
ingest_host	Required for NVIDIA NIM, Optional otherwise	Specifies the HTTP host for the service to be used for document parsing.	None	None	str	Applicable only while using NVIDIA NIM endpoints.
ingest_port	Optional Argument	Specifies the HTTP port for the service to be used for document parsing.	7670	None	int	Applicable only while using NVIDIA NIM endpoints.

The following is an example.

```
>>> from teradatagenai import VectorStore
    >>> base_dir = os.path.dirname(teradatagenai.__file__)
```

```
>>> file = os.path.join(base_dir, 'example-data', 'SQL_Fundamentals.pdf')
# Example 1: Add "LLM_handbook.pdf" to an existing'file-based' vector store.
# Create an instance of an 'file-based' vector store by passing path
# to a PDF file in "documents".
>>> vs_instance = VectorStore.from_documents(name = "vs_example_1",
                                             documents = file ,
                                             embedding = "amazon.titan-embed-text-v1")
>>> file = os.path.join(base_dir, 'example-data', 'LLM_handbook.pdf')
>>> vs_instance.add_documents(documents=file)
# Example 2: Create a new embedding-based vector store "vs_example_2"
>>> vs = VectorStore()
>>> vs.add_documents(name = "vs_example_2",
                     documents = file ,
                     embedding = "amazon.titan-embed-text-v1")
```

Adding Raw Text Using add_texts Method

The add_texts method adds raw text or list of texts to an existing content-based vector store or creates a new vector store if it doesn't exist. The syntax is

```
add_texts(texts, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the add_texts.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
name	Required Argument	Specifies the name of the vector store to be created from input list of raw text strings.	None	None	str	None
texts	Required Argument	Specifies the text or list of texts to be added to the vector store.	None	None	str, list	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
# Example 1: Add texts to an existing content-based vector store "vs_example_1"
            # Create an instance of a content-based vector store by
            # passing list of raw strings in "texts" and
            # "amazon.titan-embed-text-v1" in "embedding".
            >>> vs_instance = vs.from_texts(name = "vs_example_1",
                                            texts = ["This is a sample text.",
                                                    "This is another sample text."],
                                            embedding = "amazon.titan-embed-text-v1")
            >>> vs_instance1.add_texts(texts = ["This is a sample text1.",
                                            "This is another sample text2."])
            # Example 2: Create a new embedding-based vector store "vs_example_2".
            >>> vs = VectorStore()
            >>> vs.add_texts(name = "vs_example_2",
                             texts = ["This is a sample text.",
                                      "This is another sample text."],
                             embedding = "amazon.titan-embed-text-v1")
```

Add Tables Using add_datasets Method

The add_datasets method adds tables or DataFrames to an existing content-based vector store or creates a new vector store if it doesn't exist. The syntax is

```
add_datasets(data, **kwargs)
```

Parameters

In addition to the Common Parameter Tables, the following parameters are also used in the add_datasets.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
data	Required Argument	Specifies the table name(s)/teradataml DataFrame(s) to be indexed or added to the vector store. Teradata recommends to use teradataml DataFrame as input.	None	None	str, list, DataFrame	Multiple tables, views, or DataFrames can be included in "dataset". If a table resides in a different database, provide the fully qualified name or a DataFrame object.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
						When passing multiple tables or views, ensure each has the columns specified in "data_columns" and "key_columns". If "target_database" is not set and only a table name is supplied in "dataset", the input will be searched in the default database.
name	Optional Argument	Specifies the name of the vector store to be created from the input dataset(s).	None	None	str	None
data_columns	Required Argument	Specifies the name(s) of the data column(s) to be used for embedding generation(vectorization).	None	None	str, list	When multiple data columns are specified, data is unpivoted to get a new key column "AttributeName" and a single data column "AttributeValue".
key_columns	Optional Argument	Specifies the name(s) of the key column(s) to be used for indexing.	None	None	str, list	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
>>> from teradatagenai import VectorStore, load_data
          >>> from teradataml import DataFrame
          >>> load_data("byom", "amazon_reviews_25")
          >>> amazon_reviews_25 = DataFrame('amazon_reviews_25')
          # Example 1: Add data to an existing content-based vector store "vs_example_1"
          # Create an instance of an 'content-based' vector store by passing the 'amazon_reviews_25' table.
          >>> vs_instance1 = VectorStore.from_datasets(name = "vs_example_1",
                                                       data = "amazon_reviews_25",
                                                       data_columns = ["rev_text"],
                                                       embedding = "amazon.titan-embed-text-v1")
          >>> load_data("byom", "amazon_reviews_10")
          >>> vs_instance1.add_datasets(data="amazon_reviews_10")
          # Example 2: Create a new embedding-based vector store "vs_example_2"
          >>> vs = VectorStore()
          >>> vs.add_datasets(name = "vs_example_2",
                              data = "amazon_reviews_10",
                              data_columns = ["rev_text"],
                              embedding = "amazon.titan-embed-text-v1")
```

Add Existing Embeddings Using add_embeddings Method

The add_embeddings method adds pre-computed embeddings to an existing embedding-based vector store or creates a new vector store if it doesn't exist. The syntax is

```
add_embeddings(data, **kwargs)
```

Parameters

In addition to the Generic Parameters in the Common Parameter Tables, the following are also used in the add_embeddings.

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
data	Required Argument	Specifies the table name(s)/teradataml DataFrame(s) that are pre embedded to be indexed or to be added to the vector store. Teradata recommends to use teradataml DataFrame as input.	None	None	str, list, DataFrame	If multiple tables or views are provided, each table must include the columns specified in "data_columns" and "key_columns." When "target_database" is not set and only the table name is passed to "data," the input will be searched in the default database.
name	Optional Argument	Specifies the name of the vector store to be created from input data.	None	None	str	None
data_columns	Optional Argument	Specifies the name of the column that contains the pre embedded data.	None	None	str, list	When multiple data columns are specified, data is unpivoted to get a new key column "AttributeName" and a single data column "AttributeValue".

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
is_normalized	Optional Argument	Specifies whether the input contains normalized embeddings.	False	None	bool	None
key_columns	Optional Argument	Specifies the name(s) of the key column(s) to be used for indexing.	None	None	str, list	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
 # Load the amazon reviews embedded data.
            >>> from teradatagenai import VectorStore, load_data
            >>> from teradataml import DataFrame
            >>> load_data('amazon', 'amazon_reviews_embedded')
            >>> amazon_reviews_embedded = DataFrame('amazon_reviews_embedded')
            >>> load_data('amazon', 'amazon_reviews_embedded_10_alter')
            >>> amazon_reviews_embedded_10_alter = DataFrame('amazon_reviews_embedded_10_alter')
            # Example 1: Add data to an existing embedding-based vector store "vs_example_1"
            # Create an instance of an 'embedding-based' vector store by passing the 'amazon_reviews_embedded' table to
            # "data" and "embedding" as 'data_columns'.
            >>> vs_instance = VectorStore.from_embeddings(name = "vs_example_1",
                                                          data = amazon_reviews_embedded,
                                                          data_columns = ['embedding'])
            >>> vs_instance.add_embeddings(data=amazon_reviews_embedded_10_alter)
            # Example 2: Create a new embedding-based vector store "vs_example_2".
            >>> vs = VectorStore()
            >>> vs.add_embeddings(name = "vs_example_2",
                                  data = amazon_reviews_embedded_10,
                                  data_columns = ['embedding'])
```

Deleting a Document Using delete_documents Method

The delete_documents method removes the specified documents from a file-based vector store. The syntax is

```
delete_documents(documents, **kwargs)
```

Parameters

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
documents	Required Argument	Specifies the files to be deleted from the Vector Store. It can be used to specify input documents in file format. A directory path or wildcard pattern can also be specified.	None	Valid file paths	str, list	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
# Create an instance of an 'file-based' vector store by passing path
# to a PDF file in "documents".
# Note:
#   This is optional and can be skipped if the vector store is already created.
import teradatagenai
import os
base_dir = os.path.dirname(teradatagenai.__file__)
file = os.path.join(base_dir, 'example-data', 'SQL_Fundamentals.pdf')
vs_instance = VectorStore.from_documents(
    name="vs_example_1",
    documents=file,
    embedding="amazon.titan-embed-text-v1"
)
# Example 1: Delete "SQL_Fundamentals.pdf" from an existing 'file-based' vector store.
vs_instance.delete_documents(documents=file)
```

Deleting a Dataset Using delete_datasets Method

The delete_datasets method removes the specified datasets from a content-based vector store. The syntax is

```
delete_datasets(data, **kwargs)
```

Parameters

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
data	Required Argument	Specifies the name of the tables or teradataml DataFrames to be deleted from the VectorStore.	None	Valid table names or DataFrames	str, DataFrame, list of str/DataFrame	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
# Create an instance of an 'content-based' vector store by passing the 'amazon_reviews_25' table.
# Note:
#   This is optional and can be skipped if the vector store is already created.
from teradatagenai import VectorStore, load_data
from teradataml import DataFrame
load_data("byom", "amazon_reviews_25")
amazon_reviews_25 = DataFrame('amazon_reviews_25')
amazon_reviews_10 = load_data("byom", "amazon_reviews_10")
vs_instance1 = VectorStore.from_datasets(
    name="vs_example_1",
    data=["amazon_reviews_25", "amazon_reviews_10"],
    key_columns=["rev_id", "aid"],
    data_columns=["rev_text"],
    embeddings="amazon.titan-embed-text-v1"
)
# Example 1: Delete data from an existing content-based vector store "vs_example_1"
vs_instance1.delete_datasets(data="amazon_reviews_10")
```

Deleting an Embedding Using delete_embeddings Method

The delete_embeddings method removes the specified embedding data from an embedding-based vector store. The syntax is

```
delete_embeddings(data, **kwargs)
```

Parameters

Parameter	Required	Description	Default Values	Permitted Values	Types	Notes
data	Required Argument	Specifies the name of the tables or teradataml DataFrames containing the embedding data to be deleted from the Vector Store.	None	Valid table names or DataFrames	str, DataFrame, list of str/DataFrame	None
update_style	Optional Argument	Specifies the style to be used for alter operation of the data from the vector store when "search_algorithm" is KMEANS/HNSW.	MINOR	MINOR, MAJOR	str	None

The following is an example.

```
# Create an instance of an 'embedding-based' vector store by passing the 'amazon_reviews_embedded' and
# 'amazon_reviews_embedded_10_alter' tables to "data" and "embedding" as 'data_columns'.
# Note:
#   This is optional and can be skipped if the vector store is already created with the embedding data.
# Load the amazon reviews embedded data.
from teradatagenai import VectorStore, load_data
from teradataml import DataFrame
load_data('amazon', 'amazon_reviews_embedded')
load_data('amazon', 'amazon_reviews_embedded_10_alter')
vs_instance = VectorStore.from_embeddings(
    name="vs_example_1",
    data=['amazon_reviews_embedded', 'amazon_reviews_embedded_10_alter'],
    data_columns=['embedding']
)
# Example 1: Delete data from an existing embedding-based vector store "vs_example_1"
amazon_reviews_embedded_10_alter = DataFrame('amazon_reviews_embedded_10_alter')
vs_instance.delete_embeddings(data=amazon_reviews_embedded_10_alter)
# Example 2: Delete data from an existing embedding-based vector store "vs_example_1"
vs = VectorStore(name="vs_example_1")
amazon_reviews_embedded = DataFrame('amazon_reviews_embedded')
vs.delete_embeddings(data=amazon_reviews_embedded)
```

Errors and Limitations in Vector Store

The following sections provides the details of various error codes you may see while using vector store.

- Error Codes and Messages
- REST API Error Codes
- Limitations

Error Codes and Messages

These are the error codes and their corresponding messages which may occur when you work with vector store.

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
401, 403	API returns 401 Unauthorized or 403 Forbidden	Authentication	• User does not have the access to Vector Store Service. • PEM file and PAT tokens Used are incorrect.	• User needs to request for the right permissions, authentication tokens. • There can be different scenarios which can result into this. • ○ Make sure that correct PAT and PEM file information is passed. ○ PEM file name/path passed is correct. ○ PEM File name must be same as the PEM key name used in CCP UI. ○ When PEM is downloaded, make sure that PEM is saved, not just downloaded.	1. DataDog logs - CCP authentication logs 2. Check logs at Datainsights/VS service in DD – Add DD log query. 3. If above both not helpful - take remote of datainsights VM using teleport and check container logs - add step how to check logs into VM
504	API return 504 ERROR: The request could not be satisfied	Execution Failure	CCP timeout issue. API checks containers are unhealthy. When the database is under high load and queries are responding slowly.		

teradatagenai and teradataml Error Codes

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
TDML_2001	Following required arguments are missing: <names of the missing argument>	Argument Check	Missing required arguments.	Check the help/documentation and pass the correct type of value.	NA
TDML_2004	Argument 'arg_name' should not be empty string	Argument Check	Empty value or empty string is passed to the argument 'arg_name'.	Check the help/documentation and pass the correct type of value.	NA
TDML_2007	Invalid value(s) 'arg_value' passed to argument 'arg_name', should be: [List of permitted values].	Argument Check	Permitted value not passed.	Check the help/documentation and pass the correct type of value.	NA
TDML_2008	Invalid type(s) passed to argument	Argument Check	Incorrect type of value passed to the argument.	Check the help/documentation and	NA

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
	'<argument_name>', should be: <correct type>.			pass the correct type of value.	
TDML_2054	Current Teradata Vantage Connection context is empty or not set. Use create_context or set_context.	Connection Error	No connection to database.	Get the connection to Vantage using "create_context()" or "set_context()" Check the documentation of "create_context()"	NA
TDML_2061	Provide either 'client_id' argument(s) or 'pat_token' and 'pem_file' argument(s)	Argument Check	User has provided mutually exclusive arguments to set_auth_token() User has not provided any of the required arguments either 'client_id' or 'pat_token' and 'pem_file'	Use the 'pat_token' and 'pem_file' arguments to run set_auth_token(). Check the help/documentation for this function.	
TDML_2064	Arguments pat_token and pem_file are mutually inclusive. Please provide all arguments or None.	Argument Check	Only pat_token or pem_file arguments are used. All mutually inclusive arguments are not specified.	Specify all mutually inclusive arguments, that is, pat_token or pem_file. Check the help/documentation for this function.	
TDML_2312	Argument 'base_url' should not be ending with 'data-insights' or 'open-analytics	Argument Check	"base_url" argument passed to set_auth_token() contains the URL ending with 'data-insights' or 'open-analytics'	Remove 'data-insights' or 'open-analytics' from the end of the base URL and pass that value and rerun the command.	
TDML_2412	Failed to execute set_auth_token. Use valid value for 'base_url'.	Authentication Failure	Invalid base URL passed to the argument 'base_url'.	Recheck the base_url passed to the function and pass the correct one and run the same. Check user guide and documentation for this function. Look at the setup and installation section as well.	DataDog logs - CCP authentication service logs.
TDML_2412	Failed to execute set_auth_token. Use valid values for input arguments ['base_url', 'pat_token', 'pem_file']	Authentication Failure	Invalid values passed to the argument 'base_url' or PAT and PEM files are incorrect. User does not have the access to Vector Store Service. PEM file and PAT tokens Used are incorrect.	User needs to request for the right permissions, authentication tokens. There can be different scenarios which can result into this: 1. Make sure that correct PAT and PEM file information is passed. 2. PEM file name/path passed is correct. 3. PEM File name must be same as the PEM key name used in CCP UI. 4. When PEM is downloaded, make sure that PEM is saved, not just downloaded.	DataDog logs - CCP authentication service logs.
TDML_2412	Failed to execute disconnect. No active	Execution Failure	Observed when disconnecting the	No action needed, there is no active session to	NA

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
	database session to disconnect		VectorStore session. There is no active session hence the error.	disconnect from.	
TDML_2412TDML_2412	Failed to execute <api_name>. <Error_message from the REST API Service>	Execution Failure	Error returned from the REST APIs/Vector Store Service when executing the “api_name”. <Error_message from the REST API Service> is thrown here. Check the REST API/Vector Store service error messages.	Check the appropriate User Actions section. Otherwise, raise the concern with teradataml/teradatagenai team.	NA
TDML_2412	JSONDecodeError Failed to execute <api_name>	Execution Failure	Error returned from the REST APIs/Vector Store Service when executing the “api_name”. <Error_message from the REST API Service> is thrown here. Check the REST API/Vector Store service error messages.	Check the appropriate User Actions section for the REST API/Vector Store Service. Otherwise, raise the concern with teradataml/teradatagenai team.	
TDML_2412	Auth token is required to run VectorStore(). Set it using set_auth_token()	Authentication	User has not setup the authentication tokens.	As directed in the error set the authentication token using “set_auth_token()”. Check the help and user guide to know more about the usage of the function.	NA

AI_TextEmbeddings Error Codes

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
9134	[AWS] Request failed. Make sure Region, ModelName, and ModelArgs (if given) are correct.	Argument Check	One or more of the following arguments is invalid: Region, ModelName, and ModelArgs.	Check that the given Region exists in AWS, the given ModelName exists in Amazon Bedrock, and ModelArgs is a valid JSON payload.	
9134	[Azure] Request failed. Make sure ApiBase, ApiVersion, DeploymentId, and ModelArgs (if given) are correct.	Argument Check	One or more of the following arguments is invalid: ApiBase, ApiVersion, DeploymentId, and ModelArgs.	Check that ApiBase and ApiVersion exist in the Azure OpenAI API. Verify that DeploymentId corresponds to a model deployment in Azure OpenAI. Check that ModelArgs is a valid JSON payload.	
9134	[GCP] Request failed. Make sure Project, Region, ModelName, and ModelArgs (if given) are correct.	Argument Check	One or more of the following arguments is invalid: Project, Region, ModelName, and ModelArgs.	Verify that Project, Region, and ModelName exist in GCP. Check that ModelArgs is a valid JSON payload.	
9134	[NIM] Request failed. Make sure ApiBase, ApiKey, ModelName, and ModelArgs (if given) are correct and model service is online.	Argument Check	One or more of the following arguments is invalid: Project, Region, ModelName, and ModelArgs.	Verify that NIM service is running at ApiBase URL. Ensure ApiKey is valid. Check that ModelName exists and ModelArgs is a valid JSON payload.	

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
9134	The role does not have the required S3 permissions.	Authentication Failure	The roleName given in BatchArgs argument does not have ListBucket, PutBucket, and GetBucket permissions on the given input and output S3 buckets.	Make sure the given role has the correct S3 permissions as per Create a custom service role for batch inference - Amazon Bedrock.	
9134	Invalid AWS role. Check that it exists and has the correct trust policy.	Authentication Failure	The roleName given in BatchArgs argument does not have the correct trust relationship to call Amazon Bedrock.	Make sure the given role has the correct trust relationship to call Amazon Bedrock as per Create a custom service role for batch inference - Amazon Bedrock.	
9134	Invalid batch S3 URI argument: <S3 URI>	Argument Check	The input or output S3 URI given in BatchArgs is invalid.	Check that the given S3 URIs match ^s3://[a-z0-9][-.a-z0-9]{1,61}(?:/[-!_*'().a-z0-9A-Z]+(?:/[-!_*'().a-z0-9A-Z]+)*)?/?$.	
9134	Output S3 bucket does not exist	Argument Check	Output S3 bucket given in BatchArgs does not exist.	Make sure the given bucket exists in the AWS account.	
9134	Batch size (<# of input rows>) is less than the minimum required size (100).	Execution Failure	Bedrock batch mode requires at least 100 input rows.	Use a larger input table or on-demand inference instead.	
9134	<Parameter name> argument is not supported by the function.	Argument Check	<Parameter name> is not a valid parameter.	Check the user guide for valid parameters.	

REST API Error Codes

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
400	Failed to establish a database connection.	Network Issue or some configuration issue	Network issue or HOSTNAME_ARGS might not be pointing correctly.	Check the docker logs	DD Non Prod and DD Prod
400	Object list is available only for metadata-based vector stores.	Argument Check	Object list is available only for metadata-based vector stores.		DD Non Prod and DD Prod
400(Permissions APIs)	Invalid Action	Bad Request	Check the request	They need to check the URLs	DD Non Prod and DD Prod
400	{"detail": "Embeddings generation failed for <failedcount> rows. Please check for errors in message column in <databaseName>.<temp_embeddings_table_name> table or set ignore_embedding_errors to True and try again." }	Bad Request	Invalid data or invalid embedding parameter	They need to check the message column in the table mentioned in the error message for all the rows with NULL in the Embedding column.	
401	{"detail": "Invalid or missing credentials."}	Unauthorized	Invalid credentials	Check for credentials	
403	{"detail": "User without TD_AIAdmin role cannot terminate another user's session."} {"detail": "User must have the TD_AIAdmin role to create a new vector store."} {"detail": "User must have the ADMIN permission to grant/revoke permissions for this vector store."} {"detail": "User must have the TD_AIAdmin role to view the objects matching a pattern."}	Argument Check	User without TD_AIAdmin role cannot terminate another user's session. Or TD_AIAdmin role to create a new vector store. Or TD_AIAdmin permission to grant/revoke permissions for this vector store. Or TD_AIAdmin permission to view the objects matching a pattern.	Either the Admin User or specific user who created can terminate TD_AIAdmin can only create the Vector Store. TD_AIAdmin can grant/revoke permissions for this vector store. TD_AIAdmin can view the objects matching a pattern.	DD Non Prod and DD Prod

Error Code	Error Message	Type	Root Cause	User Actions	Logs Locations
404	Database '<database_name>' does not exist. Object '<database_name>.<object_name>' does not exist. Column '<column_name>' does not exist in the object '<database_name>.<object_name>'."	Argument Check	Incorrect Database name. The object does not exist on database. The column_name does not exist on database.	Fix the arguments as per the errors.	DD Non Prod and DD Prod
404	{"detail": "No matching rows found."}	Not Found	No details found from search.	Need to check the previous VS creation requests or need to check embeddings	DD Non Prod and DD Prod
408	{"detail": "Timeout waiting for another operation to complete for vector store <vs_name>."}	Timeout Error	Another create/update/destroy operation is in-progress for the same vectorstore Retrying creation with same name.	Wait for sometime for previous action to be over	
409	Vector store <vs_name> already exists. Please use a different name for the vector store.	Argument Check		They can try to update it.	DD Non Prod and DD Prod
409	{"detail": "Vector store <vs_name> is currently being created, updated, or destroyed."}	Conflict	Conflict requests	Wait for sometime for previous action to be over	DD Non Prod and DD Prod

Limitations

Limitations

These are the limitations of Vector store:

- The maximum sessions are 40.
- The PDF upload size is limited to 1 GB.
- When the database is under high load and queries are responding slowly, the API may temporarily return 504 Gateway Time-out errors even while your query continues running in the background. Check the status after a while when the database load has decreased.
- Special characters or keywords are not supported as database, table, or column names.
- The AITextEmbedding embeddings generation is as per rate limited by AWS / Azure per POD.
- AITextEmbedding scalability is limited to a few million rows.
- Only available in AWS Regions that support Bedrock.
- Only available in Azure Regions that support OpenAI.

Additional Information

Changes and Additions

The table lists changes and additions in this release of the Vector Store User guide.

Date	Release	Description
Aug 2025	20.00.00.06	Included details for: • Creating a vector store using From Methods • Managing vector store using add and delete methods
Jul 2025	20.00.00.05	Included support for: • File-based Vector store creation • Embedding-based Vector store • Vector store type attribute • Obtaining various details of a vector store • Nvidia NIM integration
Jun 2025	20.00.00.05	Initial version includes: • Basics of Vector Store • Creation of vector store • Administration of Vector Store • Vector Store Management

Teradata Links

Link	Description
https://docs.teradata.com/	Search Teradata Documentation, customize content to your needs, and download PDFs. Customers: Log in to access Orange Books.
https://support.teradata.com	Helpful resources in one place: • Support requests • Account management and software downloads • Knowledge base, community, and support policies • Product documentation • Learning resources, including Teradata University
https://www.teradata.com/University/Overview	Teradata education network
https://support.teradata.com/community	Link to Teradata community